

分类号 TP391 密级 公开
UDC 编号

雲南大學

碩士研究生學位論文

題 目 基于 PCI Express 的高速数据传输通道的设计与实现

Title Design and Implementation of High Speed Data Transmission Channel Based on PCI Express

学院（所、中心） 信息学院

专 业 名 称 计算机技术

研 究 方 向 FPGA 技术

研究生姓名 孙欣欣 学号 12018002078

导 师 姓 名 杨军 职称 教授

2021 年 5 月

扉页 1:

云南大学 硕士研究生学位 申请简况表	论文 预 审	论文预审结果：通过预评审			
		专家姓名	职称	所在单位（校内：学院/校外：所在单位）	
		孔兵	副教授	云南大学	
		武浩	副教授	云南大学	
	论文 送 审	专家姓名	职称	所在单位（校内：学院/校外：所在单位）	结果
		李红灵	副教授	云南大学	良好
		戴伟	副教授	昆明理工大学	良好
	论文 答 辩	答辩结果：答辩通过，同意毕业，建议授予工程硕士学位			
		答辩专家	职称	所在单位（校内：学院/校外：所在单位）	
		商云辉	高级工程 师	云南省通信管理局	
		蒋慕蓉	教授	云南大学	
		王津	副教授	云南大学	

摘要

随着电子信息的发展，现如今软件无线电系统中数据传输总线的带宽有限，软件无线电平台的通信对于各种高速数据传输的要求越来越高，这就需要选择高速串行总线进行数据传输。新的高速串行计算机扩展总线标准（Peripheral Component Interconnect Express, PCIE）相对于 PCI、USB 和以太网传统总线具有传输速率高、带宽高、可靠性高和兼容性好等优势，能够满足软件无线电的传输需求，因此设计出 PCIE 总线的高速数据传输通道具有重要意义。

本文基于 PCIE 的高速传输特性，针对软件无线电的愿景需求及应用场景，提出了一种数据传输通道的设计方案。本文在 PCIE 协议研究下首先设计了数据传输通道的总体框架，然后根据总体框架在 Xilinx 公司 K7 系列的 FPGA 开发平台上以 Verilog HDL 硬件语言来实现 PCIE 接口逻辑和 PCIE DMA 控制逻辑的设计。其中在 FPGA 逻辑设计中，针对多通道同时访问一块 DDR3，设计了一种通道轮循方式的 DDR3 存储器，解决了冲突问题；针对多通道传输的实时性要求，设计了一种命令缓冲机制的 DMA 引擎，提高了传输带宽；针对数据传输通道中速率动态可变，设计了一种动态优先级仲裁机制的通道仲裁器，优化了传输效率。

本文使用 ISE14.7 调用 Modelsim se 对 FPGA 逻辑设计进行仿真验证，并使用 Modelsim se 和 Matlab 对数据传输通道进行联合仿真验证，功能验证无误后对本设计进行性能测试。为对本设计进行性能分析，本文进行 3 方面的性能测试，经测试，数据传输通道的传输速率取得了较显著的效果，从得出的实验结果中表明 PCIE 总线的数据传输通道速度较快，性能稳定，达到了设计要求，可广泛应用于卫星遥测、无人机入侵数据获取、雷达系统等软件无线电平台的高性能数据传输系统，具有一定的应用价值。

关键词：高速数据传输；PCIE；FPGA；DMA

Abstract

With the development of electronic information, the bandwidth of the data transmission bus in the software radio system is now limited, and the communication of the software radio platform has higher and higher requirements for various high-speed data transmission, which requires the selection of a high-speed serial bus for data transmission. Compared with PCI, USB and Ethernet traditional buses, the new PCIE has the advantages of high transmission rate, high bandwidth, high reliability and good compatibility. It can meet the requirements of software radio. Transmission requirements, so it is of great significance to design a high-speed data transmission channel for the PCIE bus.

Based on the high-speed transmission characteristics of PCIE, this paper proposes a data transmission channel scheme for the vision requirements and application scenarios of software radio. This article first designed the overall framework of the data transmission channel under the PCIE protocol research, and then uses the Verilog HDL hardware language to design the PCIE interface logic and PCIE DMA control logic on the FPGA development platform of Xilinx's K7 series according to the overall framework. Among them, in FPGA logic design, for multi-channel access to a DDR3 at the same time, a channel round-robin DDR3 memory is designed to solve the conflict problem; for the real-time requirements of multi-channel transmission, a command buffer mechanism DMA is designed the engine improves the transmission bandwidth; aiming at the dynamic variable rate in the data transmission channel, a channel arbiter with a dynamic priority arbitration mechanism is designed to optimize the transmission efficiency.

This article uses ISE14.7 to call Modelsim se to simulate and verify the FPGA logic design, and uses Modelsim se and Matlab to perform joint simulation verification on the data transmission channel. After the functional verification is correct, the performance of the design is tested. In order to analyze the performance of this design,

this paper conducts three performance tests on the data transmission channel. After testing, the transmission rate of the data transmission channel has achieved a more significant effect. The experimental results show that the data transmission channel speed of the PCIE bus Faster, stable performance, meets the design requirements, can be widely used in high-performance data transmission systems of software radio platforms such as satellite telemetry, UAV intrusion data acquisition, radar systems, etc., and has certain application value.

Key Words: High-speed data transmission; PCIE; FPGA; DMA

目录

摘要.....	I
Abstract.....	III
第一章 绪论.....	1
1.1 研究背景与意义	1
1.2 研究现状	3
1.3 论文主要工作	5
1.4 论文结构及内容安排	6
1.5 本章小结	7
第二章 PCIE 协议简介	8
2.1 PCIE 协议介绍.....	8
2.1.1 PCIE 拓扑结构	8
2.1.2 PCIE 层次结构	9
2.2 PCIE 事务介绍.....	11
2.2.1 TLP 的格式	12
2.2.2 TLP 的分类	15
2.2.3 TLP 的路由与寻找规则	17
2.3 PCIE 中断方式和配置空间.....	19
2.3.1 PCIE 配置空间	19
2.3.2 MSI 中断机制	20
2.4 本章小结	22
第三章 数据传输通道的总体设计.....	23
3.1 数据传输通道的总体框架	23
3.2 FPGA 逻辑设计.....	25
3.2.1 PCIE 接口逻辑设计	26
3.2.2 PCIE DMA 控制逻辑设计	27
3.2.3 FPGA 逻辑设计的工作原理	28
3.3 数据传输通道的开发平台	30
3.4 软件无线电的应用环境	31

3.5 本章小结	32
第四章 数据传输通道的 FPGA 设计与实现.....	33
4.1 PCIE 接口逻辑的设计与实现.....	33
4.1.1 PCIE IP 核	33
4.1.2 寄存器堆模块.....	35
4.1.3 发送引擎模块.....	37
4.1.4 接收引擎模块.....	42
4.1.5 中断引擎模块.....	46
4.2 PCIE DMA 控制逻辑的设计与实现.....	47
4.2.1 DDR3 存储器	47
4.2.2 DMA 引擎	53
4.2.3 通道仲裁器.....	57
4.3 本章小结	62
第五章 实验验证与性能分析.....	63
5.1 数据传输通道的测试平台	63
5.2 数据传输通道各模块的仿真验证	65
5.2.1 DDR3 存储器模块的仿真验证	65
5.2.2 DMA 引擎模块的仿真验证	65
5.2.3 通道仲裁器模块的仿真验证.....	67
5.3 数据传输通道整体的仿真验证	67
5.4 数据传输通道的性能测试	70
5.5 本章小结	74
第六章 总结与展望.....	75
6.1 工作总结	75
6.2 后期展望	75
参考文献.....	77
攻读硕士学位期间完成的科研成果与获得奖励.....	82
致谢.....	84

第一章 绪论

1.1 研究背景与意义

随着高速数据采集技术以及高速串行总线技术的发展,人们在无线通信领域中对高速数据传输系统的性能要求越来越高^[1],高速数据传输的能力已经成为提升数据传输速率的关键性因素^[2]。

为解决高速数据传输的无线通信系统之间数据传输标准的差异,1992年,Jeon Mitola 提出了软件无线电 (Software Defined Radio, SDR) 的概念,并给出了一种 SDR 理想结构图^[3],如图 1.1 所示。

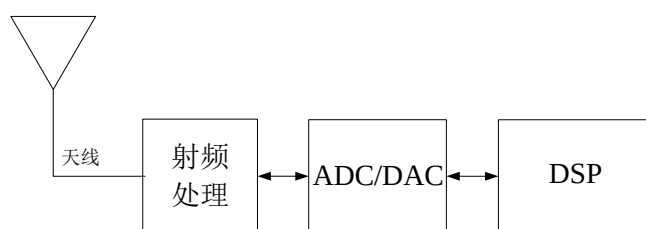


图 1.1 理想的软件无线电结构

图 1.1 中的数据流分为上传和下传。(1)上传数据时的数据流向依次通过射频处理单元、模数转换器 (Analog to Digital Converter, ADC) 和数字信号处理器 (Digital Signal Processor, DSP), 首先从天线接收到的电信号进行射频处理以输出模拟中频信号, 然后使用 ADC 对射频处理的信号做采样后将其转换为数字中频信号, 最后, DSP 对 ADC 转换后的信号执行一系列调制解调及编码。(2)下传数据时的数据流向依次通过 DSP、数模转换器 (Digital to Analog Converter, DAC) 和射频处理单元, DSP 首先对信号进行调制解调及编码以产生数字中频信号, 然后 DAC 把 DSP 接收到信号转换为模拟中频信号, 之后由射频处理对 DAC 转换的信号进行处理后输出射频信号, 最后, 将射频处理单元处理过的信号通过天线发送到空间中。

由于现阶段的 DSP 技术对图 1.1 的结构还无法满足, 这就出现了图 1.2 使用的软件无线电结构, 如图 1.2 所示为现阶段软件无线电的结构。(1)在上传数据时的数

据流向依次通过射频处理单元、ADC、下变频单元和 DSP，首先从天线接收到的信号经过射频处理后变为模拟信号，然后模拟信号经 ADC 转换后实现数字下变频，最后 DSP 将 ADC 传来的变频信号做进一步处理。(2)在下传数据时的数据流向依次通过 DSP、上变频单元、DAC 和 DSP，首先上变频单元对 DSP 发出的信号进行变频处理，然后把数字上变频信号传输到 DAC，DAC 将变频信号转换成模拟信号，最后经射频处理单元处理过的信号通过天线发送到空间中。

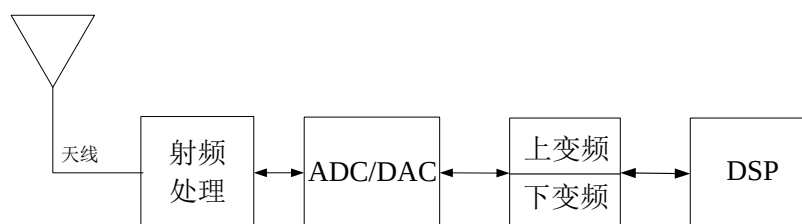


图 1.2 现阶段软件无线电结构

目前研究表明，软件无线电技术可应用于电子侦察卫星以及雷达探测和天线信号监测等领域^[4]。在电子侦察卫星场合^[5]中，为实时获取现代情报并测定辐射源的地理位置，需要高速数据传输系统将电子侦察卫星采集到的信号传输至上位机进而分析电磁信号。在无线信号监测系统^[6]中，面临实时分析处理的现场信号更为复杂，对于高速数据传输系统的要求更为严苛。在无人机定位场合^[7]中，信号需要通过视觉、射频和音频等多种手段进行测量，且飞行中无人机的信号瞬息万变，只有高速率的，大吞吐量，稳定健壮的高速数据传输系统才能满足无人机定位的应用需求。在雷达探测领域^[8]中，将采集到大量的射频信号传输至上位机进行实时处理是有非常必要。

随着现场可编程门阵列（Field Programmable Gate Array, FPGA）和 DSP 的不断发展，SDR 技术已广泛应用于无线电系统。虽然 SDR 技术特性在可编程数字信号处理平台上降低了对模拟硬件的要求，但随着 SDR 的应用越来越多，也产生了新问题：软件无线电平台上 DSP 器件的吞吐量成为无线通信算法的瓶颈。就近年对数据传输速率的相关研究与分析可知，无线局域网新标准 IEEE 802.11n 的数据传输带宽已在 4Gbps~10Gbps 范围内，最高可达 10Gbps^[9]，这也是目前无线接入设备提供的最高数据速率。同时，随着 5G 信号的逐渐发展，数据传输速率要求越来越

越高,即速率达到 1GB/s~10GB/s 的范围内才能满足硬件平台上的传输要求,此时传统的 PCI、USB 总线和以太网已不能满足现代设备的带宽、速率和可靠性数据传输的需求^[10]。随着传输数据量越来越大,通道数越来越多,对传输接口要求的速率也就越来越高,PCIE 以更高的速率,更宽的带宽和更好的扩展性可满足于软件无线电的高速数据传输需求^[11],此外,PCIE 有丰富的硬件支持,使电路结构更灵活,降低了成本,提高了速率,可以应用于个人计算机中。在此背景下研究 PCIE 总线的高速数据传输对于软件无线电平台下整体性能的提升具有实用价值,为此,本论文将对“PCI Express 的高速数据传输通道”进行研究和设计。

1.2 研究现状

FPGA 和计算机的通信问题一直是近几年的热点问题^[11],国内外研究人员和学者提出了各种接口解决方案来实现软件无线电平台通过 FPGA 与计算机之间的连接问题^[12]。国内外研究学者在软件无线电平台上对数据传输做了相关的研究工作 1986 年,美国军方开发了第一台基于软件无线电技术的通信设备^[13],1996 年,软件无线电论坛的出现使软件无线电不管对于军用还是民用都有一定的应用^[14]。随后, Lyrtech、Nutaq 和 Ettus Research 等公司推出基于软件无线电技术的通信设备^[15]。国内相对起步较晚,但在不同程度上也取了一定突破,中国的美国国家仪器有限公司^[16]研发了 HR-100 接收机和 HR-34 接收机,2018 年,张芳菊提出了一种无线接收机中高速 DMA 数据传输通道的设计^[17]。

在软件无线电平台上需通过总线接口连接外围组件进行数据传输^[18]。而相应总线的发展以及 DMA 技术的应用促进了高性能数据传输的发展。首先,I/O 总线发展历经三代,其过程为最初 ISA 为第一代 I/O 总线发展到以 PCI 为代表的第二代 I/O 总线,在 PCI、PCI-X 的基础上发展到了第三代 I/O 总线标准,目前使用 3GIO 技术的 PCIE 成为第三代 I/O 总线标准的代表^[19]。

PCIE 是第三代高速 I/O 总线,随着传输数据量越来越大,通道数越来越多,对传输接口要求的速率也就越来越高,传统的总线难以满足高速数据传输的速率需求,新一代的交换总线突破了传输带宽的瓶颈,用于互连计算和通信平台等应用程序中的外围设备^[20]。PCIE 总线与 PCI 总线相比具有以下特点:PCIE 总线为设备的连接提供了高速和高性能的串行通信,是一种基于点对点高速数据传输的

差分总线结构,而不是共享的并行总线体系结构,能够减少硬件设计的成本和复杂性^[21];PCIE总线在数据传输速率、传输带宽、可靠性等方面具有突出的优势,成为计算机连接硬盘控制器、网卡和光纤等外围设备的最佳选择,广泛应用于各种移动设备、计算机和嵌入式系统^[22];PCIE总线具有多层结构,各层均有专门协议^[23],能应用于数据传输系统的多个领域。

2001年,发布了以每通道数据传输速率为250MB/s的PCIE1.0版本。2007年,PCI-SIG在PCIE1.0的版本基础上宣布推出PCIE2.0版本,将数据传输速率提高了一倍,即每通道数据速率为500MB/s。2010年,发布了以每通道的理论传输速度为1GB/s的PCIE3.0。等到2015年,PCIE4.0发布,PCIE4.0在PCIE3.0的基础上将数据传输速率增加一倍,并能实现单通道可达2.0GB/s的传输速度。总之,PCIE概念自提出来,其数据传输速率随着版本的增加传输带宽也随之增加,使得PCIE总线在高速数据传输系统得到广泛应用。所以,本设计选择PCIE总线作为PC和FPGA之间传输的接口。

近年来国内外不少公司和机构均利用PCIE总线技术实现高速数据传输系统,并研发了高性能设备^[24]。高性能设备中国外的公司占主流,其中美国NI公司、德国Spectrum公司以及瑞典SP Devices公司等PCIE传输系统的研发处于领先地位^[25,26]。此外,国内外对基于PCIE总线的数据传输做了以下研究:国外学者Hossein等人^[27]基于Xilinx公司V7系列的开发平台上设计了基于PCIE总线的高速数据传输,并实现了DMA读速率为748MB/s和DMA写速率为784MB/s。Jose等人^[28]提出了PCIE DMA引擎,使用FPGA加速器来提高虚拟网络设备的性能。Rota等人^[29]基于FPGA提出了一种PCIE DMA架构,即DMA引擎将DMA地址列表存储在FPGA中,在减少FPGA硬件资源的基础上提高了PCIE的吞吐量。随后,国内学者高俊等人^[30]提出了一种基于PCIE的多路实时传输系统的设计,该设计通过仲裁控制实现了DDR3控制器,并在DMA传输过程中解决了拥堵问题,达到高效数据传输。2016年,电子科技大学的李文磊^[31]提出了一种基于Xilinx的ML605评估板的高速数据传输系统,该系统采用PCIE-DMA的解决方案,达到了2.0GB/s的高速数据传输的速率要求。2018年,浙江大学的陈杨^[32]基于Xilinx系列K7 FPGA的硬件开发平台上设计了PCIE以DMA的数据传输,经测试,PCIE以DMA方式进行传输的最高传输速率可达3.3GB/s。2019年,Zhao等人^[33]提出了一种利用

FPGA 内部数据缓冲存储器的资源和时间优化的 PCIE DMA 结构, 该设计通过优化 DMA 寄存器配置、避免多线程冲突来实现了高吞吐量和低延迟, 拓展了基于 FPGA 的 PCIE DMA 传输的应用。2020 年, 电子科技大学的母介滨^[34]设计了一种数据传输通道, 该数据传输通道选择 PCIE 总线作为 DMA 接口逻辑和 DMA 控制逻辑的基础, 选择 DDR3 作为数据传输时的缓存, 经测试, 本设计在单通道和多通道时的 DMA 传输速率分别可以达到 2624MB/s 和 2175MB/s。

以上这些学者和科研机构为高速数据传输通道的设计提供了硬件研究思路和研究基础。因此, 本文从提高数据传输速率的整体性能出发, 解决多通道同时访问一块 DDR3 产生的冲突问题、优化传输效率和提高传输带宽的方面进行研究与设计。

1.3 论文主要工作

本文根据 1.1 节和 1.2 节的研究意义和现状的基础上提出了一种基于 PCIE 的高速数据传输通道的设计方案。本文首先设计出数据传输通道的总体框架; 然后根据总体框架选择在 Xilinx 公司 K7 系列 FPGA 硬件开发平台上对 PCIE 接口逻辑和 PCIE DMA 控制逻辑进行设计, 其中以 FPGA 开发环境用 Verilog HDL 硬件描述语言给出 PCIE DMA 控制逻辑中 DDR3 存储器、DMA 引擎和通道仲裁器的具体设计; 最后基于 PCIE 总线的高速数据传输通道的设计在 ISE14.7 环境下进行仿真验证, 对本设计进行功能验证。同时本文对数据传输通道的进行速率测试, 实验结果表明, 本设计 PCIE 以 DMA 方式进行传输的速率较快, 适合软件无线电环境下的高速数据传输场合, 具有一定的应用价值。

因此, 论文的主要工作为:

- 1、基于 FPGA 技术, 研究和分析了 PCIE 总线的理论基础。包括 PCIE 总线拓扑结构、TLP 格式、分类和路由方式、PCIE 设备的配置空间和 MSI 中断方式。为攻克数据传输通道设计中的重点与难点奠定基础, 同时也为本文的性能指标、外围电路设计和硬件的实现提供了理论支持。

- 2、硬件设计部分: 用 FPGA 自顶向下的模块化思路实现 PCIE 接口设计和 PCIE DMA 逻辑设计, 其中以 FPGA 开发环境用 Verilog HDL 硬件描述语言对 PCIE 接口逻辑进行 RTL 设计, 从而完成了 PCIE DMA 控制逻辑与 PCIE 总线之间的交互,

进而实现 PCIE 接口通信的功能;在 PCIE DMA 控制逻辑中使用 RTL 设计了 DDR3 存储器,解决了多通道同时访问 DDR3 时出现的冲突问题,从而实现 8 通道对 DDR3 的读写操作,同时为优化通道的传输效率,本文使用 Verilog HDL 硬件语言设计了一种根据优先级选择通道的通道仲裁器,为提高数据传输带宽,使用 Verilog HDL 硬件语言设计并实现了一种命令缓冲机制的 DMA 引擎。

3、仿真验证及性能分析。基于数据传输通道的 FPGA 设计在 ISE14.7 进行仿真验证后,下载到 Xilinx K7 开发板中,然后评估数据传输通道设计中 DDR3 存储器模块、DMA 引擎模块和通道仲裁器模块以验证数据传输中多通道的冲突问题和数据传输效率是否满足设计要求,评估数据传输通道的整体设计是否能够完成既定的工作任务,最后通过对 PCIE 以 DMA 方式进行数据传输时的速率基本达到 PCIE 2.0 x8 总线的理论传输速率可知,本设计具有较高的传输速率,综合性能得到了提升。

1.4 论文结构及内容安排

第一章为本文绪论部分,其内容首先介绍了论文的研究背景及意义,然后对软件无线电平台和 PCIE 总线的研究现状进行阐述,最后简要介绍了本文的主要内容与章节安排。

第二章为数据传输通道设计的理论基础。本章对本设计中使用到 PCIE 协议的相关内容介绍,为本文后续的设计工作提供理论支撑。

第三章就如何通过 PCIE 总线进行数据传输以满足本设计的参数要求。本章首先对本设计的总体框架进行了介绍,其次对 FPGA 逻辑设计进行模块划分,对其原理进行分析,然后根据本设计需求选择了 Xilinx K7 开发板进行数据传输通道的 FPGA 设计,最后对本设计应用环境的一种无线信号接收机的结构进行介绍。

第四章为本文的核心章节,利用 FPGA 对基于 PCIE 总线的高速数据传输通道中的模块进行设计,用 FPGA 自顶向下的模块化思路实现 PCIE 接口设计和 PCIE DMA 逻辑设计,在 PCIE IP 核基础上使用 Verilog HDL 硬件语言对 PCIE DMA 接口逻辑进行设计,实现了 PCIE 接口通信的功能;在 PCIE DMA 控制逻辑使用 RTL 设计了 DDR3 存储器,解决了 8 通道中的多通道同时访问 DDR3 时出现的冲突问题,从而实现 8 通道对 DDR3 的读写操作,同时为优化通道的传输效率,本文设

设计了一种根据优先级选择通道的通道仲裁器，为提高数据传输带宽，设计并实现了一种命令缓冲机制的 DMA 引擎。

第五章为数据传输通道的实验分析。本文首先对数据传输通道的 FPGA 逻辑设计调用 Modelsim 进行仿真验证后烧入 Xilinx K7 开发板，对其进行功能验证。随后对 DMA 传输带宽进行性能测试，经测试，本设计在单通道和 8 通道时的最高传输带宽分别为 2830MB/s 和 2325MB/s，均取得了较显著的效果。

最后一章总结了在数据传输通道的设计中本人完成的成果，并为后继的研究人员的对数据传输速率的研究工作提供一些思路。

1.5 本章小结

本章首先从课题的研究背景与意义展开论述，然后对软件无线电平台和 PCIE 研究现状进行了综述，指出了高速数据传输通道进行研究的必要性，最后对论文的主要工作及内容安排和相关章节进行了介绍。

第二章 PCIE 协议简介

本文把 PCIE 作为数据传输通道的接口，为保证高效稳定运行，需对 PCIE 总线协议进行研究，明确数据传输通道的设计原理，以发挥 FPGA 硬件逻辑设计的最大性能。本章为 PCIE 协议中与数据传输通道设计相关的内容，分别从 PCIE 总线的拓扑结构、层次结构以及事务层的路由方式、PCIE 设备配置空间的组成和 PCIE 总线的中断机制进行介绍，明确本设计的内容和相关要求，为后续的设计提供了理论基础。

2.1 PCIE 协议介绍

本节分别从 PCIE 拓扑结构和层次结构对 PCIE 协议进行介绍。

2.1.1 PCIE 拓扑结构

PCIE 总线与总线共享式通讯方式的 PCI 不同，PCIE 由点到点的链路组成^[35]，并采用树形拓扑结构，如图 2.1 所示，PCIE 拓扑结构体系由 CPU、根复合体（Root Complex，RC）、端点设备（Endpoint，EP）和交换器（Switch，SW）等 PCIE 设备组成。

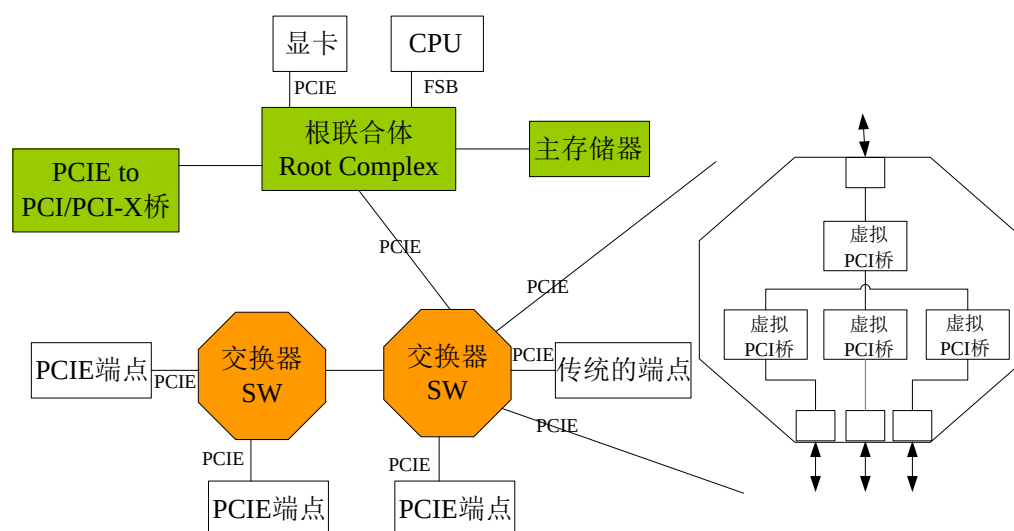


图 2.1 PCIE 总线的拓扑结构图

1. RC

如图 2.1 所示, RC 由 FSB-to-PCIE 桥和存储器组成, 且通过 PCIE 连接到 SW 上, 之后由 SW 连接 PCIE 端点从而实现了与 CPU、PCIE 总线和主存储器的通信^[36]。RC 功能有两个, 一是接收 CPU 的 IO 指令然后生成 PCIE 报文, 另一个是接收来自设备的 PCIE TLP 报文, 对 PCIE TLP 报文解析后发送给 CPU 或内存。同时, RC 作为 PCIE 传输时的请求者支持配置请求和 I/O 请求的生成, 其中, I/O 请求的位置分为 80h 和 84h, 但是 RC 不考虑根端口 PCIE 到 PCI/PCI-X 桥的 I/O 解码配置。

2. SW

SW 主要负责 PCIE 设备的消息路由, 并使用 PCIE 到 PCI/PCI-X 桥中的 PCI 桥接机制转发事务, 是 PCIE 的转接器设备, 目的是扩展 PCIE 总线^[37], 如图 2.1 所示的右半部的 SW 定义为多个虚拟 PCI-to-PCI 桥接设备的逻辑组合。SW 必须在任何端口集合之间转发所有类型的 TLPs, 且在交换器的端口中仅有一个端口指向根联合体, 并以配置软件的形式出现。SW 为下面的 Endpoint 或 SW 提供路由转发服务。

3. Endpoint

端点设备是 PCIE 事务的请求者或完成者^[38]。因 PCIE 设备的配置空间支持 Type0 型和 Type1 型配置空间, 而端点设备带有 Type 00h 配置空间头函数, 则端点设备支持配置请求。PCIE 端点设备支持 MSI 中断或 MSI-X 中断, 并且端点设备支持 64 位消息地址版本的 MSI 能力结构, 在本设计 PCIE 请求中断时满足了 MSI 中断条件。端点设备和 RC 组成一个点对点的连接从而完成 PCIE 事务传输, 实现端点设备和主存储器之间的数据传输功能。

4. PCIE 到 PCI/PCI-X 桥

PCIE 到 PCI/PCI-X 桥在 PCIE 结构和 PCI/PCI-X 层次结构之间提供连接。其基本功能为 PCIE 总线与 PCI 总线之间的转换提供桥梁。

2.1.2 PCIE 层次结构

为理解 PCIE 总线协议, 将 PCIE 分层, PCIE 层次结构如图 2.2 所示。PCIE 的层次结构模型由硬件实现, 从而保证了 PCIE 数据传输的稳定性, 提高了 PCIE 数据传输速度^[39,40]。在 PCIE 总线中, PCIE 使用数据包在 PCIE 层次结构中的各层

之间通信，数据包在接收和发送过程中需经过 PCIE 层次结构中的物理层、数据链路层和事务层。发送端的数据包首先在 PCIE 设备核 A 中生成，然后通过图 2.2 中的事务层、数据链路层和物理层再到达 PCIE 设备核 B 从而完成整个数据的传输；相反，接收端的数据包首先通过图 2.2 物理层、数据链路层和事务层，然后将数据包送入 PCIE 设备核 B 中从而完成整个数据的传输。在数据传输时本设计的上传和下载对应于 PCIE 层次结构的在接收和发送过程。

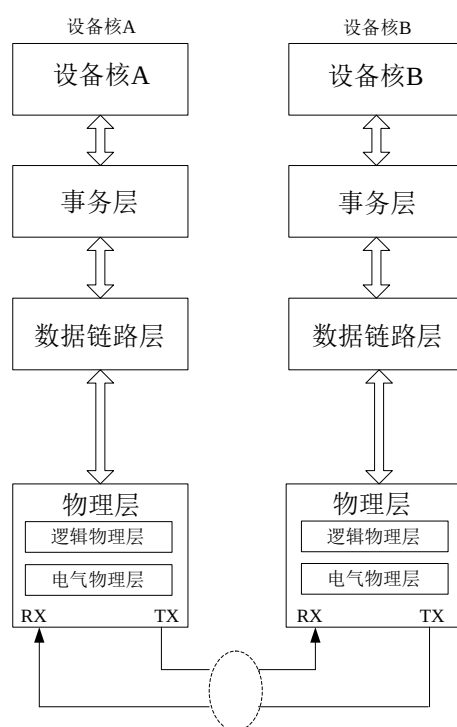


图 2.2 PCIE 总线的层次结构图

PCIE 总线基于数据包进行数据传输，并根据图 2.2 拓扑结构的 PCIE 协议标准将数据包分为三种^[41]，TLP 由事务层生成，DLLP 由数据链路层生成，物理层 PLP 由物理层生成，如图 2.3 所示为 PCIE 数据包的分层封装图。

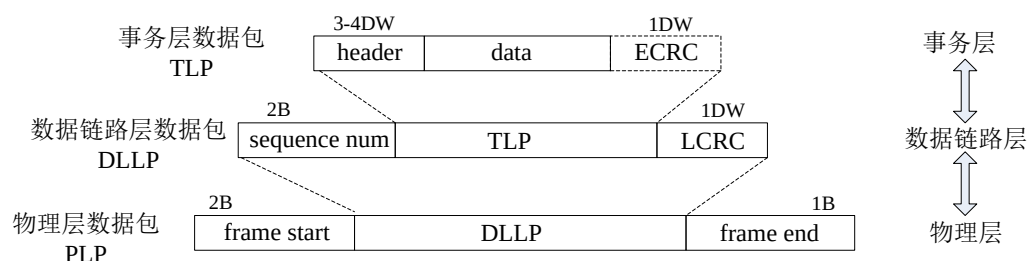


图 2.3 PCIE 数据包的分层封装图

1. 事务层

事务层主要功能是接收、缓存和传输数据包，主要职责是组装和拆卸 TLP，

定义要发送 TLP 的类型及格式，并按照 TLP 的类型分别发送到 PIO 通道和 DMA 通道中。事务层有虚拟通道管理流量控制、数据包队列管理和提供服务质量功能，以及事务顺序、电源管理和配置寄存器，还负责为 TLP 基于信用的流控制管理，其流控制管理中当接收设备事务层中的虚拟通道缓冲区还有接收 TLP 的空间，发送设备才可以在 PCIE 链路上发送相应 TLP，以保证数据传输的正确性。

2.数据链路层

数据链路层处于 PCIE 总线拓扑结构的 1 和 3 之间，主要功能是为 TLP 传输提供可靠支持，主要职责通过 PCIE 协议来控制数据传输。数据链路层对事务层的数据包进行处理，在图 2.2 中，数据包从设备 A 开始传递，事务层的 TLP 传输到数据链路层生成 CRC 并添加序列 ID，此时对应用数据保护代码和 TLP 序列号提交给物理层，以便在链路上传输。此外，数据链路层检查接收到 TLP 的完整性，检查完成后提交给事务层，以保证来自事务层的 TLP 在 PCIE 链路上的数据能正确传输。

3.物理层

物理层由两层组成，如图 2.2 所示，由图可知这两层分别为逻辑物理层和电气物理层。物理层包括 PCIE 接口操作的所有电路，还包括与 PCIE 接口初始化和维护相关的逻辑函数，并且物理层中的介质与可靠的物理环境可为 PCIE 总线的数据传输所用。物理层从 2 中接收到的信息转换成适当的序列化格式，目的是为了能通过 PCIE 链路与另一端设备以合适的带宽进行传输，则该层提供信号带宽的计算公式由文献[61]可知 PCIE 2.0 在 x8 带宽通道模式下，每条数据链路的传输速率可以达到 5GB/s，但由于采用 PCIE2.0 x8 采用 8B/10B 编码方式，实际传输速度为 4GB/s。

2.2 PCIE 事务介绍

PCIE 总线在本设计中的任务就是数据传输^[42]，PCIE 总线事务在图 2.1 中的传输过程为：

- 1、事务在 PCIE 设备之间传输时需经过 SW 交换器传输到另一个 PCIE 端点设备或者 RC 上；
- 2、RC 也可与 1 中相同的事务来访问 PCIE 设备。

基于以上 2 点，本设计中的 PCIE 总线完成了 PCIE 设备与 SW、Root Complex 以及 SW 与 SW 间的传递。

PCIE 设备通过事务完成了请求者和完成者之间的信息传输，事务的具体传输过程如图 2.2 所示的层次结构图，即从设备 A 到设备 B，或者从设备 B 到设备 A。PCIE 总线的事务在拓扑结构中实现了数据之间的传输。

PCIE 进行数据传输时按照数据交互的目的定义了四个地址空间，在地址空间的基础上定义了四种事务类型，即不同地址空间的事务类型分为以下四种：(1)存储事务；(2)配置事务；(3)I/O 事务；(4)消息事务。以上这四种事务类型在本设计中都有其独特用途，本设计根据数据传输时 PCIE 设备的接收方是否发送完成包，将 PCIE 协议中的事务分为：(1)转发事务(posted)；(2)非转发事务(non-posted)。表 2.1 给出了 PCIE 协议事务层的各种事务，并介绍了相对应的 TLP 包和完成包的类型，与这些地址格式和相关的 TLP 格式的使用规则将在本节后进行描述，以下重点介绍本设计所涉及事务协议。

表 2.1 PCIE 总线事务列举

PCIE的事务类型	TLP的类型	完成包的类型
Memory Read	MRd	Cpld
Memory Write	MWr	无
Memory Read Lock	MRdLK	cpldDLK/cplLk
IO Read	IORd	cpld/cpl
IO Write	IOWr	cpl
Configuration Read	CfgRd	cplD
Configuration Write	CfgWd	cpl
Message	Msg	无
	MsgD	

2.2.1 TLP 的格式

事务由请求和完成组成，并使用数据包进行通信^[43]，由上面所述 PCIE 进行数据传输时需依次通过事务层、数据链路层和物理层。此外，TLP 为数据传输通道中 PCIE 开发的第一手数据，并且在第五章对 FPGA 逻辑设计进行仿真验证和性能分析时，分析 TLP 是本设计中解决问题最直接的方法。

PCIE 的 TLP 大致构架由 TLP 头标、数据有效载荷和“TLP Digest”三部分组成，如图 2.4 所示，TLP 格式在 PCIE 总线的作用是在串行互连上起到了优化性能。

其中，TLP 头标在 Byte0-ByteJ 的位置处，其包括发送者的路由信息、目标地址和 TLP 总线事务类型以及数据长度等；数据有效负载在 ByteJ-ByteK-4 处，是用显示在左上角的最低地址字节来进行描述的，且由 TLP 的事务类型决定是否携带有效数据；“TLP Digest”字段是一个可选项，是否需要由 TLP 头标决定。

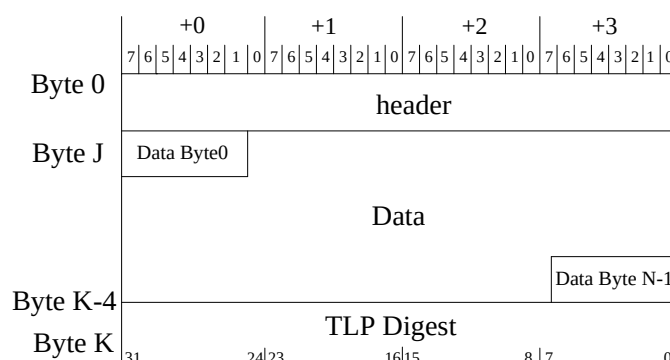


图 2.4 TLP 在不同分层上的帧格式

由图 2.3 中 PCIE 数据包的分层封装图的基础上，并在 PCIE 协议中 TLP 在不同分层上的帧格式得出如图 2.5 所示的 TLP 的基本格式，由图可知，TLP 在 PCIE 层次结构中的传输过程为：首先数据链路层在接收到事务层发送的 TLP 后，在 Header+data+ECRC 的基础上加上 Sequence number 和 LCRC，从而构成 DLLP，，然后物理层接收到数据链路层发送到 DLLP 后，在 DLLP 的基础上加上 start 和 end，从而构成 PLP。

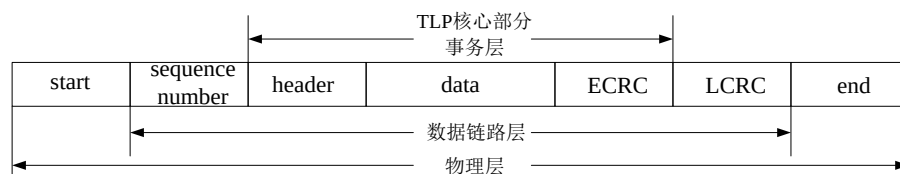


图 2.5 TLP 的基本格式

根据 TLP 的基本格式可知，TLP 头标(header)由 3 个 DW 或者 4 个 DW 组成。存储器读写 TLP 读写地址模式根据 TLP 头标长度为 3DW 还是 4DW 决定支持 32 位还是 64 位。

在 TLP 头标中第一个 DW 的 TLP 头如图 2.6 所示，由 TP、Attr 和 TH 以及 EP 等字段组成。



图 2.6 通用 TLP 头标格式

下面对图 2.6 所示通用 TLP 头标格式的各字段的含义进行介绍。如表 2.2 所示。

表 2.2 PCIE 事务列举

通用TLP头字段	含义
Fmt	TLP头标大小
Type	TLP事务类型
TC	TLP传输类别
AT	TLP地址转换
TH	TLP是否含有TPH
EP	TLP中的数据是否有效
TD	TLP中的TLP Digest是否有效
Attr	TLP的属性
Length	TLP有效负载的大小

表 2.3 TLP 头标中 Fmt[2:0]与 Type[4:0]字段

TLP 类型↵	Fmt[2:0]↵	Type[4:0]↵	TLP 头格式↵	描述↵
MRd↵	0b000↵	0b000000↵	3DW 或者↵	存储器读请求↵
	0b001↵		4DW 不带数据↵	
MWr↵	0b010↵	0b000000↵	3DW 或者 4DW	存储器写请求↵
	0b011↵		带数据↵	
Cpl↵	0b000↵	0b01010↵	3DW 不带数据↵	完成报文↵
CplD↵	0b001↵	0b01010↵	3DW 带数据↵	完成报文↵
CfgRd0↵	0b000↵	0b00010↵	3DW 不带数据↵	配置 0 读请求↵
CfgWr0↵	0b010↵	0b00100↵	3DW 带数据↵	配置 0 写请求↵
Msg↵	0b001↵	0b10xxx↵	4DW 不带数据↵	消息请求↵
MsgD↵	0b011↵	0b10xxx↵	4DW 带数据↵	消息请求↵

在 TLP 头标中，Fmt 字段和 Type 字段一起确定当前 TLP 使用的是存储事务、配置事务以及 I/O 事务还是消息事务，如表 2.3 所示。其中，Fmt[2:0] 字段用于确定 TLP 的格式、TLP 的头标和有效负载；Type[4:0] 字段规定了本设计的 TLP 事务类型。

2.2.2 TLP 的分类

2.2.1 节的表 2.2 给出了由 Fmt 和 Type 字段确认的 TLP 类型，则 2.2.2 节将由 Fmt 和 Type 字段确认的 TLP 类型按照功能分为以下 3 种：(1) 存储器读写请求 TLP；(2) 配置读写请求 TLP；(3) 完成 TLP。同时，TLP 传送方式根据 PCIE 协议中的 non-posted 事务和 posted 事务将 TLP 类型分为 non_posted 以及 posted TLP^[44]。下面分别介绍存储器读写 TLP、配置读写 TLP 和完成 TLP。

1. 存储器读写 TLP

PCIE 协议中，表 2.1 中存储事务中的 MRd 使用 non_posted 数据传输方式，同时，需要返回 Cpld。而表 2.1 中存储事务中的 MWr 则采用 posted 数据传输方式，此时，不需要返回 Cpld。

在本设计的 PCIE 事务中，存储器读写请求 TLP 所使用 TLP 头标的结构是相同的，如图 2.7 所示为存储器读写请求 TLP 的头标格式。

	+0								+1								+2								+3							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Byte 0	Fmt		Type				R	TC	R	Attr	R	TH	TP	EP	Attr	AT	Length															
Byte 4	Requester ID												Tag				Last DW BE				Frist DW BE											
Byte 8	Address[63:32]																															
Byte 12	Address[31:2]																														R	

图 2.7 存储器读写请求 TLP 头标格式

(1)、Length 字段在存储器读请求 TLP 和存储器写请求 TLP 具有不同的含义，其含义分别为事务请求方读回的数据长度和当前报文携带有效数据载荷的大小。

(2)、Last DW BE[3:0] 字段和 First DW BE[3:0] 字段在图中存储器读写请求 TLP 头标格式的 Byte 大小为 4bit 位宽，且在 PCIE 的数据传输过程中对 Length 字段的

DW 字节起到使能作用。

(3)、Requester ID 字段包含数据传输时接收方的 PCIE 设备总线号和功能号以及设备号，其用于表示事务的请求者，以便完成 TLP 进行 ID 路由。

(4)、Tag 字段为 PCIE 数据传输时请求者发出的请求序号。其为使数据传输过程中的 PCIE 设备能分析出 TLP 报文目的地址，需要将 Requester ID 字段、Tag 字段进行组合，组合后的 Transaction ID 在 PCIE 数据传输时起到了一定的作用。

(5)、Address 字段中包括 Address[63:32]字段和 Address[31:2]字段，其中 Address[31:2]字段可从 PCIE 总线的发送引擎和接收引擎两个方面进行解释说明。

2.配置读写请求 TLP

PCIE 的配置读写请求 TLP 由 RC 发起，PIO 方式读写，每个 PCIE 设备都有属于自己的配置空间，并支持 Type00h 和 Type01h 两种配置请求，即 PCIE 设备的配置空间中存储着设备的属性和配置信息，配置读写 TLP 的头标格式如图 2.8 所示。

	+0								+1								+2								+3							
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
Byte 0	Fmt		Type				R	TC	R	Attr	R	TH	TP	EP	Attr	R	Length															
Byte 4	Requester ID												Tag				Last DW BE		Frist DW BE													
Byte 8	Bus Number				Device Number		Fuction Number		R		Ext Reg Number		Register Number		R																	

图 2.8 配置读请求 TLP 头标格式

由于配置读写 TLP 的配置请求报文使用 2.2.3 节所述的 ID 路由，即没有 Address 字段，除了以下这些字段，图中的 TC、TH、Attr 以及 AT 和 Length 均为固定值。

(1)、Requester ID 字段用于事务发起方的 PCIE 设备的总线号、设备号和功能号。

(2)、Fuction Number 和(1)和(2)两个字段一起组成 Completer ID，其中，(1)Bus Number; (2)Device Number 字段。Completer ID 用来标识配置事务的响应者，并存储着 TLP 访问目标设备的相应标号。

(3)、Register Number 字段是寄存器号，Ext Reg Number 字段是扩展寄存器号。

PCIE 配置空间时的地址偏移由这 Register Number 字段和 Ext Reg Number 字段两个字段进行控制。

3.完成 TLP

PCIE 总线中的完成 TLP 分为 CplD 和 Cpl 两种，是以存储器读请求、I/O 读写请求和配置读写请求的响应数据包，其中 Cpld 根据目的不同将 Cpld 分为两种：1、non-posted 请求的 TLP。2、posted 请求的 TLP。如图 2.9 所示是完成包 TLP 的头标格式。

	+0								+1								+2								+3										
	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0			
Byte 0	Fmt			Type				R	TC		R	Attr	R	TH	TP	EP	Attr	AT		Length															
Byte 4	Completer ID														Compl status		B	C	M	Byte Count															
Byte 8	Requester ID														Tag						R	Lower ADDRESS													

图 2.9 完成包 TLP 头标格式

(1)、Requester ID 字段和 Tag 字段一起组成 Transaction ID，实际上本设计是由 Transaction ID 字段值进行数据传输。

(2)、Completer ID 字段为数据传输时完成者的标识符，而且与 Requester ID 一起实现 TLP 的路由并结束总线事务。

(3)、Compl status 字段表明数据传输时请求事务的完成状态。

(4)、Byte Count 字段记录完成全部 PCIE 数据传输时需要读取的有效数据。

(5)、Lower Address 字段表示目前完成 TLP 的有效数据对应地址的最低几位。

2.2.3 TLP 的路由与寻找规则

PCIE 协议制定了一套严密的路由与寻址规则^[45]，保证了本设计在事务层发起的 PCIE 事务能完成正常的传输，TLP 路由方式实质为 TLP 在图 2.1 的基础上进行传输时使用哪条路径到达 Endpoint 或 Root Complex 的一种方法，其传输过程如图 2.10 所示。此外，本文在表 2.1 总结了 PCIE 的总线事务，根据 PCIE 的总线事务类型本文对应了四种不同类型的地址空间，为了进一步表述地址空间，PCIE 总线

设置了三种路由方式：(1)地址路由；(2)ID 路由；(3)隐式路由。如表 2.4 为各种事务包路由方式。

表 2.4 事务层包的路由方式

TLP类型	路由方式
存储器读写事务、锁定存储读事务	地址路由
I/O读写事务	地址路由
不带数据/带数据的消息事务	地址路由/ID路由/隐式路由
不带数据/带数据的完成事务	ID路由
Type0、Type1配置读事务 Type0、Type1配置写事务	ID路由

如图 2.10 所示为 TLP 路由方式的传送过程。

由图可知，当 TLP1 发送数据包时，RC 首先对 TLP1 的物理地址转化为映射地址，然后 TLP1 经过 PCI Bus0 发往 SW 后，由 SW 检查 TLP 数据包是否有错误，最后由 P-P1 根据内部路由表中的信息决定该 TLP 是否被接收，以上就是 TLP1 的路由过程。

TLP2 的路由过程与 TLP1 的路由过程相似，TLP2 的路由过程为 EP2 经过 PCI Bus3 并根据 P-P3 内部路由表中的信息进而传送到 RC 上的数据包，而 TLP2 主要是图 2.10 中的存储器进行访问。

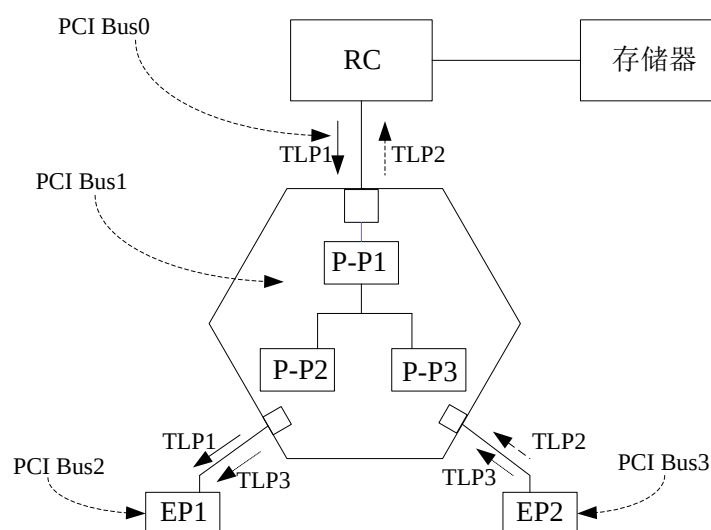


图 2.10 TLP 的路由方式

1.地址路由

在 PCIE 总线中，存储器读写 TLP、I/O 读写 TLP 均使用 TLP 路由方式中的地

址路由，并使用 TLP 中的 Address 字段实现路由选径，地址路由的 TLP 中包含目的设备的地址。其中，MW_r TLP 的整个传输过程：当 PCIE 设备执行 MW_r 时，Root Complex 会把 Address 指向的映射地址映射到目标设备的物理内存上。

2.ID 路由

PCIE 总线协议规定配置请求事务和完成包事务使用 ID 路由的方式，即基于 ID 路由的 TLP 使用总线号、功能号和设备号进行路由选址后确定目标设备的位置。且为支持 ID 路由，每个 PCIE 设备功能中的寄存器为从 TLP 头标中获取总线号、设备号和功能号，则需要包括存储总线号以及设备号和功能号。

由于配置读写请求采用的是 ID 路由，而地址路由需在操作系统配置后才能进行选择，基本配置完成后，如果 RC 想要访问 EP1，只需发送配置请求事务，由图 2.10 可知此时 TLP 携带的 RC_Transaction_ID 被送到 EP1 上，为使地址路由传输过程中的 Cpld 能正确到达 Root Complex，EP1 在响应 Cpld 时需把 RC_Transaction_ID 填写在 TLP 的相应位置中。

3.隐式路由

PCIE 协议使用中断请求、总线事务锁定以及电源管理和错误信息等事件来表达消息 TLP，PCIE 的消息 TLP 来自 Root Complex 或广播报文并使用隐式路由的 Route 字段的值选择路由方式。

2.3 PCIE 中断方式和配置空间

本节分别从 PCIE 配置空间和中断方式对 PCIE 协议进行介绍。

2.3.1 PCIE 配置空间

每个 PCIE 设备都具有 PCIE 的配置空间，PCIE 设备配置空间由配置寄存器组成，且配置空间的大小和结构是固定的，PCIE 设备与 PCI 设备的配置空间的区别在于新的配置寄存器和能力^[46]。PCIE 总线规范定义了 Type0 型和 Type1 型两种 PCIE 端点设备配置空间，Type1 型的 PCIE 端点设备配置空间主要包括本设计与 2.2.3 节 TLP 路由方式相关的配置寄存器，而 Type0 型的 PCIE 端点设备配置空间根据上一节介绍的 PCIE 协议分为以下 3 部分组成：

1、PCI 头标区；

2、PCI 设备专用寄存器组和新功能寄存器组；

3、PCIe 扩展配置空间。

如图 2.11 所示为 PCIe 配置空间分布图，下面对这三部分进行介绍。



图 2.11 PCIe 配置空间分布图

1. PCI 头标区

PCI 头标中的 PCI 头标区由 16 个双字组成，位于 4KB 配置空间的低地址处。其作用是为了 PCIe 总线能兼容 PCI 总线。

2. PCI 设备专用寄存器组和新能力寄存器空间

本区域在 1 和 3 结构之间，并比 PCI 头标区多 32DW，即本区域为 48DW。其中的设备 ID 标记 PCI 厂商识别码，厂商 ID 标记 PCI 装置识别码，设备 ID 和厂商 ID 用于识别 PCIe 设备，由图可知存放着基地址寄存器 0~基地址寄存器 5，这 6 个寄存器加之设备 ID 和厂商 ID 可唯一确定 PCIe 端点设备。

3. PCIe 扩展配置空间

本区域位于 2 的结构之下，大小为 960DW，这些新能力寄存器结构以单向链表的形式进行级联，并以 Capability 结构的形式出现在配置寄存器中。

2.3.2 MSI 中断机制

PCIe 设备向处理器提交 MSI 中断请求，这离不开 PCIe 设备配置空间中的 MSI Capability 结构。PCIe 协议定义了 MSI Capability 结构格式如图 2.12 所示。

MSI 中断机制中的存储器写请求 TLP 使用的地址和数据负载分别为图 2.12 中位于 094h~098h 的 Message Address 和 09Ch 的 Message Data 字段。下面对 MSI Capability 结构中的各个字段进行说明。

MSI Control [↗]	Nextcap [↗]	MSI Cap [↗]	090h [↗]
Message Address(Lower) [↗]			094h [↗]
Message Address(Upper) [↗]			098h [↗]
Reserved [↗]		Message Data [↗]	09Ch [↗]
Mask Bits [↗]			0A0h [↗]
Pending Bits [↗]			0A4h [↗]

图 2.12 PCIE 的 MSI Capability 能力结构

(1)MSI Control 字段为 PCIE 的中断状态信息,MSI Control 字段的结构如表 2.6 所示。

表 2.6 MSI Control 字段的结构

Bits	定义
8	Masking Capable
7	64bit Address Capable
6:4	Multiple Message Enable
3:1	Multiple Message Capable
0	MSI Enable

(2)MSI Cap 字段为 MSI Capability 的 ID 号,本设计选择 MSI Capability 的 ID 号的值为 0x05。

(3)Message Address 字段为用于存放 PCIE 数据传输过程中产生 MSI 中断时 TLP 的目的地址。

(4)NextCap 字段保存 PCIE 数据传输时的下一个 MSI Capability 地址信息。

(5)Message Data 字段用于保存 PCIE 数据传输过程中产生 MSI 中断时 TLP 的数据负载。

(6)Mask Bits 字段的 32 位对应于 PCIE 数据传输时产生的 32 个 MSI 中断请求的屏蔽开关位。

(7)Pending Bits 字段与 Mask Bits 字段一样具有 32 位,与 Mask Bits 不同的是

对应于 PCIE 数据传输时的 32 个 MSI 中断请求。

其中 Pending Bits 字段与 Mask Bits 字段在 PCIE 数据传输时一同使用以防止处理器丢失 MSI 中断请求。

2.4 本章小结

本章首先对 PCIE 的拓扑结构和层次结构进行介绍，然后介绍了事务层的 TLP 数据包以及事务层的三大路由方式，最后阐述了 PCIE 设备的配置空间和 MSI 中断方式。该章为本文设计奠定了理论基础。

第三章 数据传输通道的总体设计

本章在第二章介绍 PCIE 协议的基础上设计了一种数据传输通道。本章首先介绍了数据传输通道的总体框架，在总体框架基础上重点介绍 FPGA 逻辑设计的方案，然后在数据传输通道的硬件开发平台上选择 Xilinx K7 开发板进行 FPGA 逻辑设计，最后介绍了数据传输通道在应用环境中的总体结构。下面内容将介绍本文如何通过 PCIE 总线进行数据传输以满足本设计的要求。

3.1 数据传输通道的总体框架

本设计的数据传输通道旨在完成上传数据和下传数据时的两个功能。1、在上传数据时首先将 FPGA 中数据缓存到 DDR3 存储器，然后上位机请求后把数据从 DDR3 存储器中取出并通过 PCIE 总线将数据传输到上位机；2、在下传数据时上位机中的数据首先通过 PCIE 总线下传至 FPGA，FPGA 读取这些数据并存储到 DDR3 存储器中。

基于以上两点，加上本设计要将软件无线电平台所获取多通道中的数据传输到上位机进行实时处理和分析^[47]。即在数据传输通道的设计中需要解决以下问题：

- （1）DDR3 作为数据传输通道的缓存区，当多通道同时访问一块 DDR3 存储器时，出现冲突问题；
- （2）PCIE 传输时不牺牲吞吐量的前提下优化传输效率；
- （3）PCIE 下传数据量大，板卡与计算机交互频繁；而上传数据量小、延迟小，实时性要求高，导致传输带宽慢。

对于以上 3 个问题，本文分别对上传数据通道和下传数据通道的设计进行介绍，为使本设计的数据传输通道能达到高速数据传输效果，PCIE 以 DMA 方式在 FPGA 与计算机之间进行数据传输，FPGA 与计算机之间的数据传输由 FPGA 逻辑进行调控，即本设计采用模块化设计的思想，并使用 FPGA 技术作为一种基于 PCIE 的高速数据传输通道的解决方案。

数据传输通道的总体框图如图 3.1(a)所示，本文的主要工作如图 3.1(b)所示，本文为实现数据传输过程中对大量数据进行缓存，需在 FPGA 逻辑设计前添加 DDR3 存储器；为实现设备对数据传输和处理进行控制，需在 DDR3 数据缓存后

设置 FPGA 逻辑设计进行控制；为实现 FPGA 和主机之间的传输，需连接 PCIE 总线；为实现对整个传输通道设计的启动和测试以及数据的显示，需在计算机中进行上位机显示。

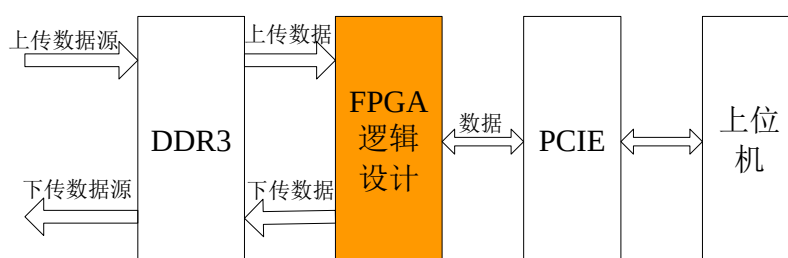


图 3.1(a) 数据传输通道总体框架

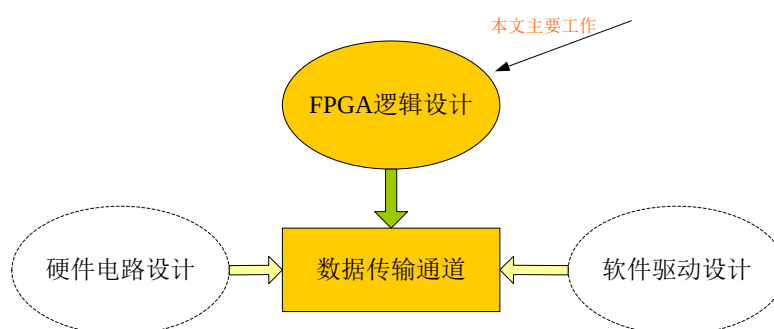


图 3.1(b) 数据传输通道的主要工作

PCIE 高速传输前的缓存采用 Xilinx K7 系列 FPGA 芯片的解决方案^[48]。目前，在 PCIE 总线的高速数据传输领域中的数据缓存使用 FPGA 内部的存储器资源远远不够，因此，本设计为保持数据传输通道的传输速度和传输质量，选择存储容量为 MB 的 DDR3。DDR3 从 2009 年以来一直是主流的高速内存设备，从早期传输速度以 KB/s 为单位到现在以 GB/s 为单位，存储速度上有了不同数量级的变化^[49]，并且使用了 8 个逻辑库的设计，因此内存容量更大，满足于数据传输通道时的缓存^[50]。所以本文使用 DDR3 来实现数据传输通道的缓存功能。

FPGA 逻辑设计主要控制 PCIE 总线的高速数据传输通道过程中的数据传输，本设计通过 DMA、PIO 两种方式完成数据传输。数据传输通道在传输过程中对数据量大、带宽高和实时性高的数据，本文使用 DMA 方式，而对于数据量小且不需要连续传输时，本文将使用 PIO 方式。DMA 数据传输的实现主要是对传输控制器

中的相关寄存器进行操作，而 CPU 只需要在数据传输开始和结束时进行相应的处理，从而减少了 CPU 的消耗时间。DMA 传输时首先申请一条 DMA 通道，即通道应用程序在外部设备打开时完成，并在外部设备关闭时释放，如果外部设备需要发送数据，则系统需要打开一个缓冲区来存储数据，传输完成后外设需发送中断函数，进而通知 CPU 数据已发送完成并释放对总线的控制。PIO 传输时首先向内存中写入数据，PCIE 设备会向 CPU 请求一个中断，然后 CPU 通过 PCIE 总线将该 PCIE 设备的数据读取到 CPU 内部的寄存器，最后再把数据从内部寄存器写入到内存中从而完成一次 PIO 传输。

PCIE 总线作为 FPGA 和 PC 间的传输介质，连接 FPGA 和 PC，基于 PCIE2.0 总线在单通道时的传输速率最高可达 5Gbps^[51]，而本设计中与上位机通信时选用 PCIE 接口的通道数为 8，所以能达到本设计的传输要求。如图 3.1 (b) 所示，上传数据时，FPGA 逻辑设计首先将 DMA 数据包和命令信息从 DDR3 存储器中读出，再通过 PCIE 总线发送给计算机，最后将数据写入内存。而下载数据时，PCIE 总线将上位机中内存的数据传输到 FPGA 逻辑设计中，并用 DDR3 存储器将其数据进行缓存。

上位机由内存和 CPU 组成，是整个性能测试和功能验证过程中人机交互的界面^[52]，对接收到的数据进行解析并对界面化的实时显示。FPGA 板卡通过 PCIE 接口与计算机连接^[53]，并在计算机上验证数据传输通道传输数据时的正确性和稳定性。

3.2 FPGA 逻辑设计

根据 3.1 节数据传输通道的总体设计可知，FPGA 逻辑设计为本文的设计重点，即对数据传输通道的时序和逻辑进行控制，实现数据传输通道的功能逻辑，使得各个模块高效有序地工作。如图 3.2 所示为 FPGA 逻辑设计的总体框图，本文数据传输通道的 FPGA 逻辑设计由 PCIE 接口逻辑和 PCIE DMA 控制逻辑两部分组成，橘色部分为 PCIE DMA 控制逻辑，其它为 PCIE 接口逻辑。即 DDR3 存储器模块、DMA 引擎模块和通道仲裁器模块组成 PCIE DMA 控制逻辑；PCIE IP 核模块、寄存器堆模块、发送引擎模块、接收引擎模块和中断引擎模块组成 PCIE 接口逻辑，下面本设计将从 PCIE 接口逻辑、PCIE DMA 控制逻辑和工作原理三方面对数据传

输通道的 FPGA 逻辑设计进行介绍。

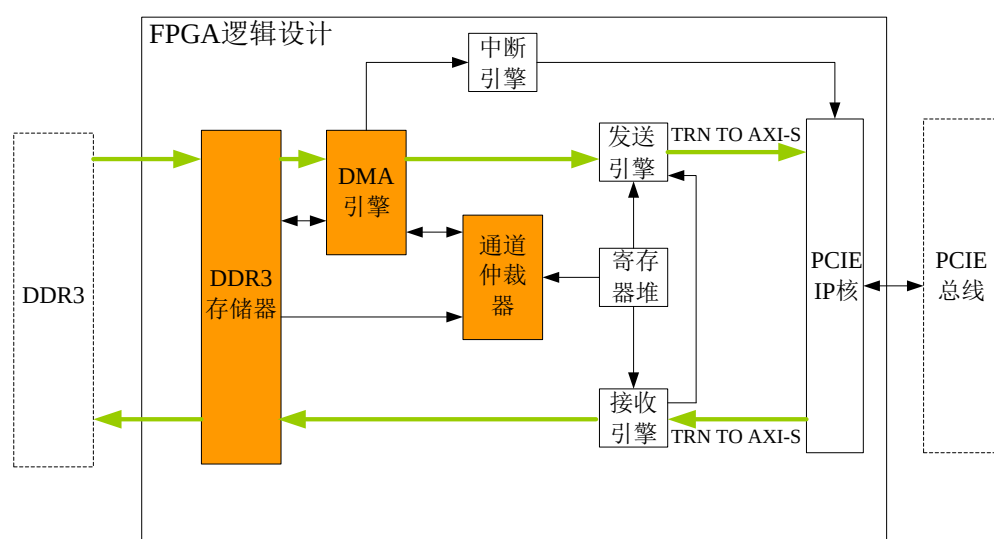


图 3.2 FPGA 逻辑设计总体框图

3.2.1 PCIE 接口逻辑设计

本设计在第二章 PCIE 协议的基础上用 PCIE IP 核对 FPGA 和 PCIE 总线进行交互,此时 PCIE 接口逻辑作为 PCIE DMA 控制逻辑与 PCIE 间的交互介质,使 PCIE DMA 控制逻辑的设计独立于 PCIE 总线之外,进而实现 PCIE 接口通信的功能。PCIE 接口逻辑由 PCIE IP 核、发送引擎模块、接收引擎模块以及寄存器堆模块和中断引擎模块组成。

PCIE IP 核调用了 Xilinx FPGA 芯片内部的 PCIE 硬核,根据本设计需求在 Xilinx K7 中提供的 IP 核基础上对 PCIE 接口逻辑进行设计。

寄存器堆模块包括 DMA 数据传输时各通道的状态信息,还有与之相关的寄存器,但本设计访问这些寄存器需要通过 PIO 方式。

发送引擎模块和接收引擎模块由 FPGA 编程进行设计,设计过程中产生 TLP 包的逻辑和解析 TLP 包的逻辑的读 TLP 请求和写 TLP 请求、读 TLP 解析和写 TLP 解析、CPLD 解析,并负责处理 FPGA 逻辑设计与 PCIE IP 核接口之间的数据,其中的数据包括 FPGA 逻辑设计产生的 TLP 以及计算机产生的 TLP。

中断引擎模块根据 FPGA 逻辑设计的发送引擎和接收引擎状态判断各通道中 DMA 传输是否完成,完成后向计算机发起 MSI 中断请求。

3.2.2 PCIE DMA 控制逻辑设计

PCIE DMA 控制逻辑采用模块化设计思想,以 Verilog HDL 语言完成了各个通道的数据搬移、对 DDR3 的访问以及通道仲裁,从而确保 PCIE DMA 控制逻辑能正确发送和接收数据。PCIE DMA 控制逻辑由 DDR3 存储器、DMA 引擎和通道仲裁器组成,下面介绍这三个模块的功能。

DDR3 存储器为 PCIE 进行数据传输时方便使用各个通道中的数据从而使用了 DDR3 读写接口,FPGA 逻辑设计中的各个模块通过 DDR3 存储器进行访问。DDR3 存储器内部根据本设计通道的需求设置了 8 个通道缓存区,从而支持多通道同时访问 DDR3 存储器,通道中的数据由接收引擎通过 PCIE 以 DMA 方式写入 DDR3 存储器,或者通过 PCIE 以 DMA 方式从 DDR3 存储器中读出来,再通过通道将数据发送出去。本文为解决多通道同时访问 DDR3 产生的冲突问题,设计了一种通道轮循的 DDR3 存储器。

DMA 引擎根据 PCIE 接口中的多个 DMA 申请传输时由图 3.2 中通道仲裁器对多通道同时访问时的仲裁结果,并根据寄存器堆中 DMA 传输过程中产生的信息将通过 PCIE 总线向 DDR3 存储器的数据缓存区搬移数据,或者从 DDR3 存储器的缓存区向 PCIE 总线搬移数据。DMA 引擎工作过程是由 DMA 引擎的 FIFO 与本设计的 PCIE IP 核形成流水线结构来完成数据传输,其中 DMA 引擎的 FIFO 包括读/写数据 FIFO 和命令 FIFO,读/写数据 FIFO 和命令 FIFO 各自负责 DMA 引擎中的传输数据和传输命令。本设计因 Xilinx K7 提供的 PCIE IP 核并没有实现本设计达到 DMA 引擎逻辑的功能,所以 DMA 引擎的控制逻辑需要在 FPGA 硬件开发平台上编写 Verilog HDL 代码来实现其功能。

通道仲裁器旨在当多个通道同时申请数据传输选择其中一个通道进行 DMA 传输。本设计的通道仲裁器采用基于信号带宽的动态优先级仲裁策略,将 8 通道的仲裁请求按照动态配置的优先级进行分组,每组通过仲裁器进行仲裁产生该组的结果,并根据仲裁结果按照优先级等级的顺序依次向各个 DMA 引擎的通道发送仲裁授权,从而实现了通道数据的传输。

的分类选择出此次数据传输的 TLP 类型，即存储器写请求 TLP，并完成 AXI-S 的接口时序转换后传输给 PCIE IP 核。

2、下传数据和上传数据的传输过程类似，即数据依次通过 IP 核、接收引擎和 DDR3 存储器完成传输。下传数据时首先由 DDR3 存储器监测 DDR3 中 8 通道的数据缓存数量，DDR3 存储器从通道仲裁器中读取通道号进行数据传输的条件为 8 通道中的任一通道对应缓存区剩余缓存空间的内存足够容纳一次 DMA 传输的数据，其次通道仲裁器根据信号带宽的动态优先级仲裁策略选择出 8 通道中的一个通道，并选择出将要进行数据传输的通道号以及寄存器堆中读出的 DMA 传输长度、传输地址组成符合 PCIE 协议的报文格式输出给 DMA 引擎，然后 DMA 引擎根据通道仲裁器输出的通道号以及相关信息组成存储器读请求 TLP 头标，同时，发送引擎根据存储器读请求 TLP 的头标和数据负载组成存储器读请求 TLP，并完成 AXI-S 的接口时序转换后发送给 PCIE IP 核，之后上位机的 CPU 对 PCIE IP 核发来的存储器读请求 TLP 进行回应，且回应的 TLP 类型为存储器读完成 TLP，最后接收引擎收到上位机对 PCIE IP 核回应的 TLP 后从中提取出数据，并发送给 DDR3 存储器中相应的 FIFO 缓存区。

在实现 DMA 传输进行上传和下传的同时，本设计依然采用 PIO 的传输方式，即在 FPGA 逻辑设计中使用 PIO 的传输方式对寄存器堆模块进行读写控制，其中包括 PIO 传输时对配置空间的访问以及读写开始命令和寄存器的初始化等。

1、本设计当上位机采用 PIO 传输方式写寄存器时，即在 PIO 传输时 PCIE IP 核首先收到寄存器写请求 TLP，然后接收引擎对其 TLP 进行解析，之后根据解析的寄存器写请求 TLP 提取出写地址/数据，最后向寄存器堆模块中写入数据。

2、本设计当上位机采用 PIO 传输方式读寄存器时，即在 PIO 传输时 PCIE IP 核首先收到寄存器读请求 TLP，然后接收引擎对其 TLP 进行解析，之后根据解析的寄存器读请求 TLP 提取出读地址/数据，最后发送给发送引擎后形成寄存器读完成 TLP，并将其传递给 PCIE IP 核。

FPGA MK7325FA 开发板配备了型号为 MT41K256M16TW-107: P 的 4 片镁光 DDR3 内存, DDR3 内存频率 1600MHz, 带宽可达 $1600\text{MHz} * 64\text{bit}$ 。因单片 DDR 内存的大小为 512MB, 则四个 DDR3 内存总共为 $512\text{MB} * 4 = 2\text{GB}$, 其 FPGA MK7325FA 开发板带有的 DDR3 的数据接口为 16bit, 可满足本设计的大容量缓存。

3.4 软件无线电的应用环境

本设计的数据传输通道在软件无线电平台上达到了高速传输的效果, 在软件无线电平台的实际使用中, 数据传输的通道数和 DMA 传输长度可由用户通过 FPGA 开发板带有 PCIE 预留的接口进行配置^[55], 本文为达到软件无线电传输过程中带宽高, 通道数多的需求, 设计了一种基于 PCIE 总线的高速数据传输通道, 即在本设计中将通道数设置为 8, 其中包含 8 路上传数据通道和 8 路下传数据通道。

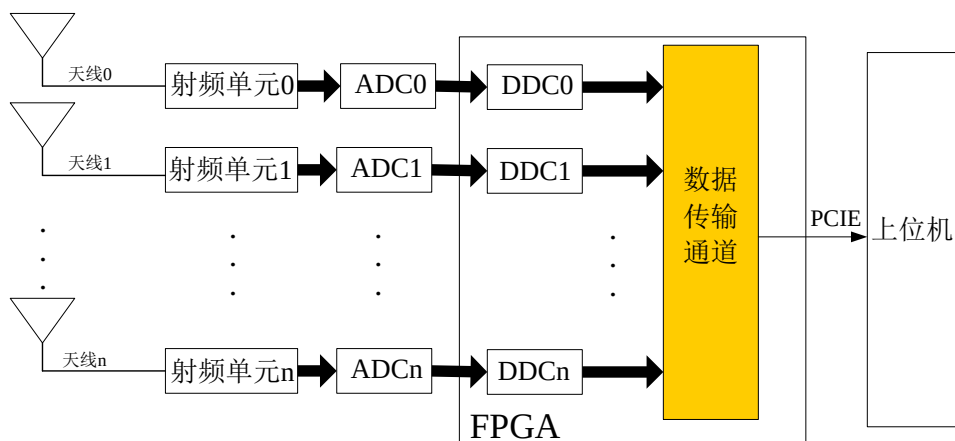


图 3.5 多通道无线接收机传输模型

本设计基于软件无线电平台的多通道无线接收机传输模型如图 3.5 所示, 该无线接收机具有 n 条通道, 其中 $n=8$, 每条通道由射频单元、ADC 和 DDC 组成, 且每条通道接入数据传输通道, 通过数据传输通道上传至上位机, 此外, 该模型旨在第五章对本设计进行整体仿真验证。基于数据传输通道的设计中在软件无线电使用过程中的数据传输速率接近 36Gbps, 本文的数据传输接口选择 x8 通道的 PCIE 接口^[56]。对于第一章介绍的 PCIE 2.0 的理论传输速率为 5Gbps, 则 x8 为 $5\text{Gbps} * 8 = 40\text{Gbps}$, 所以 x8 通道的 PCIE 接口即可满足数据传输的需求。

3.5 本章小结

本章首先对数据传输通道的总体设计进行了介绍，其次对数据传输通道中各个模块的设计和 FPGA 逻辑设计的原理进行介绍，然后在 FPGA 硬件开发平台上选择 Xilinx K7 开发板进行 FPGA 逻辑设计，最后对数据传输通道应用环境的一种无线信号接收机的结构进行介绍。

第四章 数据传输通道的 FPGA 设计与实现

本文第三章为数据传输通道的总体设计，并介绍了 FPGA 逻辑设计的各个模块模和FPGA逻辑设计的工作原理，然后根据软件无线电的需求将通道数设置为8。本章对数据传输通道的各个模块进行 FPGA 的设计与实现，由第三章可知 FPGA 逻辑设计包括 PCIE 接口逻辑设计和 PCIE DMA 控制逻辑设计，下面对这两部分设计进行详细说明。

4.1 PCIE 接口逻辑的设计与实现

PCIE 接口逻辑作为 PCIE DMA 控制逻辑与 PCIE 间的交互介质，使 DMA 控制逻辑的设计独立于 PCIE 总线之外，进而实现 PCIE 接口通信功能。PCIE 接口逻辑由 PCIE IP 核、寄存器堆模块、发送引擎模块以及接收引擎模块和中断引擎模块组成，下面对五个模块进行 FPGA 的设计与实现。

4.1.1 PCIE IP 核

本节在 Xilinx 7 系列 IP 核的功能特点和 PCIE IP 核的参数设置基础上，借助于 K7 开发板上的 FPGA 芯片内部 PCIE 硬核实现了本设计数据传输通道的基本功能，为本设计提供基础。本设计依据 3.2 节和 3.3 节的内容使用了 Xilinx K7 中提供 PCIE IP 核的功能接口完成了 PCIE 接口逻辑的设计。PCIE IP 核的框架如图 4.1 所示，是 7 系列 FPGA 嵌入式 PCIE 硬核，封装了 PCIE 协议的事务层、数据链路层和物理层的一部分。

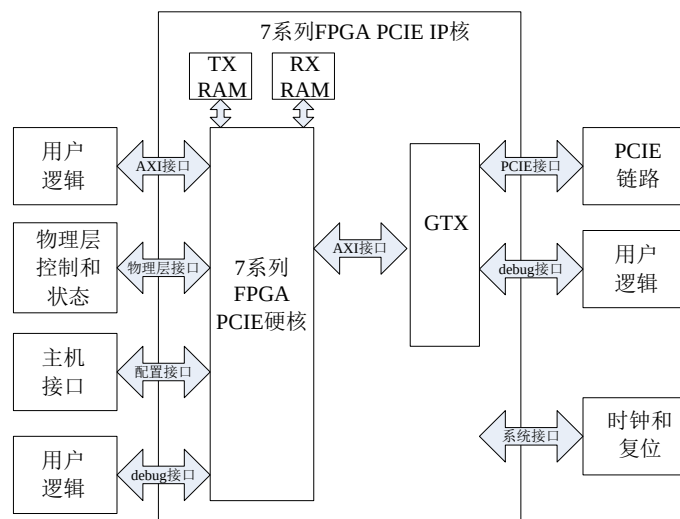


图 4.1 PCIE IP 核结构框图

通过 ISE14.7 中的 IP 核例化工具对 PCIE IP 核进行例化处理，根据本设计的硬件需求，对 IP 核的参数进行设置并赋予一定的说明。

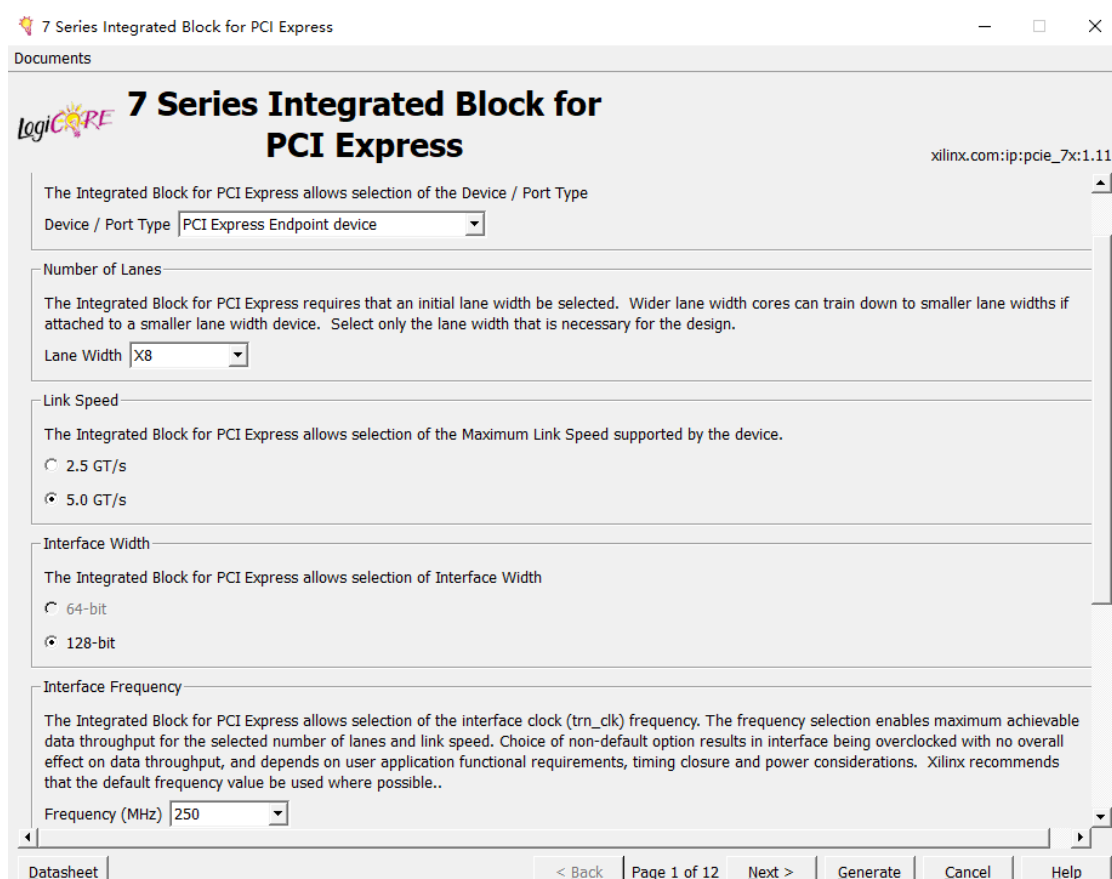


图 4.2 PCIE IP 核基础配置界面

在图 4.2 中 PCIE IP 核的配置界面为配置 PCIE 设备的基本功能，由图可知，

Port Type 设置了 PCIE IP 核的端口类型; Number of Lanes 设置了 PCIE 支持的最大通道数, 本文选择 x8 通道; Link Speed 为 PCIE 最大链路传输速率, 本文选择 5.0GT/s 的传输速率; Interface Frequency 为本设计 AXI-S 接口的时钟频率, 本文选择 250MHz; Interface Width 为本设计 AXI-S 接口的数据位宽, 本文选择 128bit。

Xilinx K7 系列的 PCIE IP 核根据 PCIE 配置空间的结构可知最多支持 6 个 BAR 寄存器, 本设计使用两个寄存器: (1)BAR0; (2)BAR1。

如图 4.3 所示为 PCIE BAR 空间的配置界面, 由图可知, BAR0 Options 为配置 PCIE BAR0 的空间, 本文选择的类型为存储器型, 内存大小 2KB 和寻址位宽为 32bit; BAR1 Options 为配置 PCIE BAR1 空间, 本文选择的同 BAR0 都为存储器型、内存大小为 64KB 和寻址位宽为 32bit。由于 BAR0 空间与 BAR1 空间在本设计中所要存储的内容不一样, 所以本文在 PCIE 传输时用 BAR0 和 BAR1 存放的寄存器分别为与 DMA 传输相关和与设备状态相关。

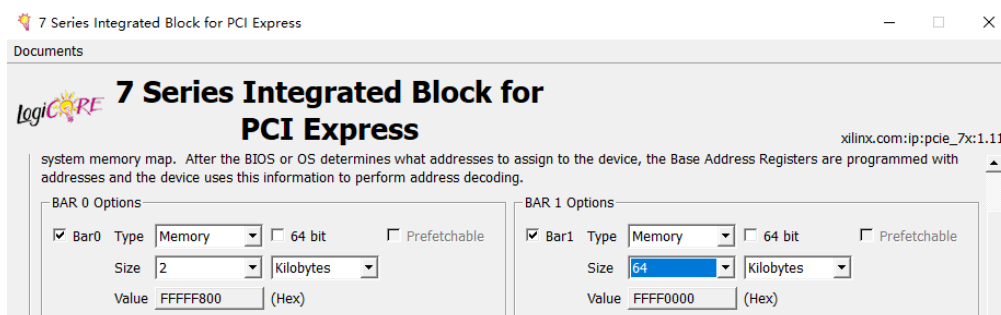


图 4.3 PCIE IP 核 BAR 空间配置界面

4.1.2 寄存器堆模块

寄存器堆模块为数据传输通道设计的信息集合地。在 PC 端, 上位机驱动程序可读写本设计中所用到的寄存器来配置和控制 PCIE DMA 控制逻辑。同时, 在 FPGA 板卡端的各模块可通过寄存器来获取数据。

如表 4.1 为寄存器堆模块的信号及定义表。发送/接收引擎通过以地址线加数据线形式给出的接口信号连接寄存器堆模块, 同时, 通道仲裁器等模块通过具体信号线形式给出的接口信号连接寄存器堆模块。

表 4.1 寄存器堆的接口信号及定义

端口名称	输入\出	描述
wr_en	Input	写寄存器接口
wr_addr	Input	
wr_data	Input	
rd_en	Input	读寄存器接口
rd_data	Output	
rd_addr	Input	
dma_size	Output	TLP大小
dma_en_chn(n\0~7)	Output	使能信号
dma_done_chn(n\0~7)	Input	结束标志
dma_bw_chn(n\0~7)	Output	信号带宽

寄存器堆的空间结构图由 A、B、C、D 四部分组成，如图 4.4 所示，其中 A 为端点设备信息、B 为通道 0~7 写寄存器区、C 为通道 0~7 读寄存器区、D 为通道 0~7 状态区组成。

端点设备信息在空间结构中 0x00 的位置处，其中包括 FPGA 型号、版本号以及 IP 核数据位宽等硬件设备信息，并且端点设备所述的信息用于 5.1 节数据传输通道的测试平台的上位机软件的驱动程序的初始化。

通道 0~7 写寄存器区在空间结构中 0x04~0x80 的位置处，其中包括写 DMA 起始地址、写 DMA TLP 传输次数、写 DMA 使能以及写 DMA 结束等写寄存器信息，并且该写 DMA 寄存器区域对数据传输通道的写 DMA 过程进行控制。

通道 0~7 读寄存器区在空间结构中 0x84~0xF0 的位置处，其中包括读 DMA 起始地址、读 DMA TLP 传输次数、读 DMA 使能以及读 DMA 结束等读寄存器信息，并且该读 DMA 寄存器区域对数据传输通道的读 DMA 过程进行控制。

通道 0~7 状态区在空间结构中 0xF4~0x11 的位置处，其中包括本设计所使用的带宽、数据源使能以及频点和增益等状态信息，并且该状态区数据传输通道中的状态由信息决定。

地址	寄存器名	31	24	23	16	15	7	0
0x00	端点设备信息	FPGA型号		PCIE IP核数据位宽		版本号		保留
0x04 ... 0x80	通道0~7写 DMA寄存器区	写DMA起始地址					保留	
			写DMA TLP传输次数					
		保留					写DMA使能	
		保留					写DMA结束	
0x84 ... 0xF0	通道0~7读 DMA寄存器区	读DMA起始地址					保留	
			读DMA TLP传输次数					
		保留					读DMA使能	
		保留					读DMA结束	
0xF4 ... 0x11	通道0~7读状 态区	保留			带宽	保留	数据源使能	
		频点						
		增益						

图 4.4 寄存器堆空间结构图

4.1.3 发送引擎模块

发送引擎模块在数据传输过程中需要与 4.1.1 节 PCIE IP 核的发送接口信号进行连接，从而实现了 TLP 数据包发送给 PCIE IP 核。下面介绍发送引擎模块。

根据 4.1.1 节对 PCIE IP 核协议进行解释，并从 3.2.3 节了解到本文数据传输时不仅包括端点设备和主机间的 DMA 方式，还包括读写访问存储空间的 PIO 模式，但不管是 DMA 方式的传输还是 PIO 功能的命令，发送引擎模块的事务包类型都有存储器读/写请求 TLP 以及存储器读完成 TLP。所以发送引擎模块的工作过程为：

1、对于以 DMA 方式进行的数据传输，发送引擎首先接收来自 4.2.2 节 DMA 引擎在 PCIE 进行数据传时产生的 TLP 头标和 TLP 数据负载，然后按照 PCIE 在数据传输通道的相应功能并根据 TLP 头标和数据负载对存储器读请求 TLP 和存储器写请求 TLP 进行组包，最后将组成的 TLP 通过 TRN TO AXI-S 发送给 PCIE IP 核的接收接口。

2、对于以 PIO 方式进行的数据传输，发送引擎则读出存储在 4.1.2 节寄存器堆模块中的数据，然后将读出后的数据组成存储器读完成 TLP，最后组成的 TLP 通过 TRN TO AXI-S 发送给 PCIE IP 核的接收接口。

如图 4.5 所示，发送引擎模块的逻辑设计由发送接口状态机和事务类型仲裁逻辑两部分组成。发送接口状态机开始运行是由以下两者决定：(1)DMA 引擎中命令

FIFO 和数据 FIFO 中的状态信号；(2)接收引擎模块中存储器读完成 TLP 的请求信号。在状态机启动后将 DMA 引擎的 TLP 发送给接收引擎。事务类型仲裁逻辑的功能为多个 TLP 同起请求时根据 2.2 节的 PCIE 事务中三类 TLP 的优先级决定响应顺序。

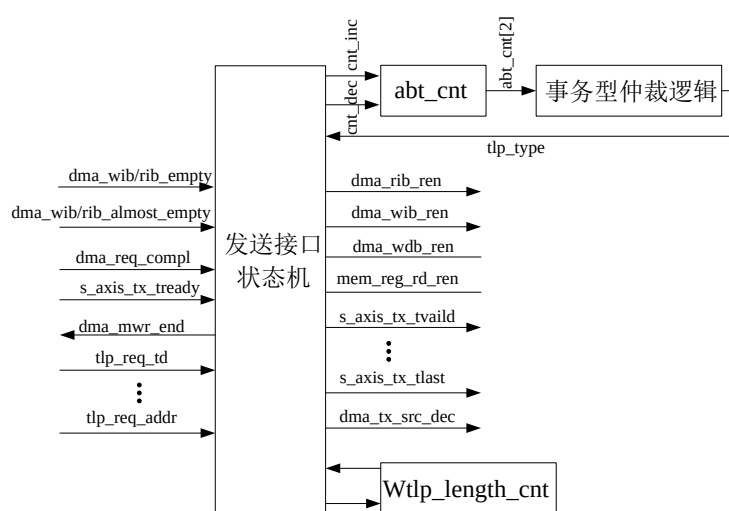


图 4.5 发送引擎模块结构图

表 4.2 发送引擎的接口信号及定义

端口名	输入\出	描述
tx_tready	Input	AXI-S的s_axis的接收端口
tx_tdata	Output	
tx_tkeep	Output	
tx_tlast	Output	
tx_tvalid	Output	
tx_src_dsc	Output	
dma_compl	Input	TLP完成包的端口
dma_compl_wd	Input	
dma_compl_done	Output	
dma_req_tc	Input	存储器读请求TLP头标的关键字段
dma_req_td	Input	
dma_req_ep	Input	
dma_req_len	Input	
dma_req_tag	Input	
dma_req_addr	Input	
dma_wib_empty	Input	DMA写FIFO的端口
dma_wib_almost_empty	Input	

下面在介绍发送引擎模块状态机之前，先用表 4.2 发送引擎模块的接口信号及定义进行说明。

发送接口状态机主要作用是 TLP 组包，主要功能是完成从其他模块中读取数据。存储器写请求 TLP 和存储器读请求 TLP 的数据从以下两部分缓冲区中读出。

(1)4.2.2 节 DMA 引擎中的 A 缓存区以及 B 缓存区，A 为 wdma_info_buffer、B 为 wdma_data_buffer 缓存区。(2)4.1.2 节寄存器堆的 C 缓存区，C 为 reg_buffer。发送引擎模块通过如图 4.6 所示的状态机实现发送接口的逻辑功能，发送接口的状态机的三个状态分别为，BMD_TX_DMA_RST 状态、BMD_TX_DMA_MWR_QW 状态和 BMD_TX_DMA_MWR_QW 状态。这三个状态的定义及转移过程如下：

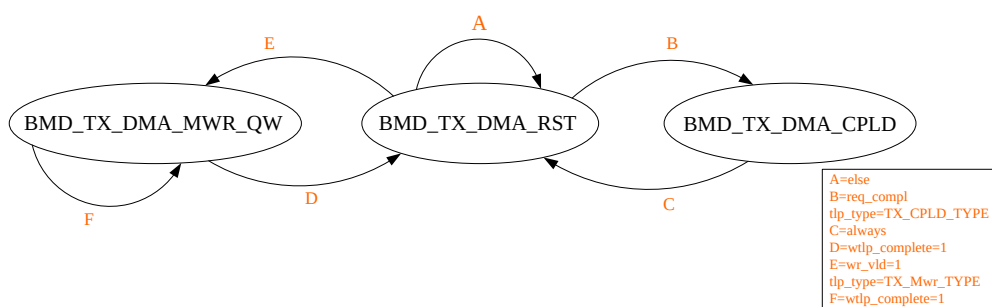


图 4.6 发送接口的状态转移图

BMD_TX_DMA_RST 状态：该状态为初始状态，各信号复位，等待发送 TLP，复位后状态机根据 dma_wib_empty、dma_rib_empty、dma_req_compel 和 s_axis_tx_tready 等控制信号来判断当前发送的 TLP 类型。在对 DMA 写操作判断时需要增加对 PCIE IP 核发送就绪标志的 s_axis_tx_tready 有效判断，在发送 FIFO 可读数据达到 8 个以上，PCIE IP 核的发送缓冲区至少能接收一个 TLP 的有效信号 s_axis_tx_tready。

(1)若此时发送的是存储器读请求 TLP，则信号 s_axis_tx_vaild 和信号 dma_rib_ren 的值等于 1，为获取存储器读请求 TLP 头标信息，在数据传输时提取出 dma_rib_dout 信号的信息，因存储器读请求 TLP 的头标长度为 3 DW，所以 PCIE 发送数据包结束标志的信号 s_axis_tx_tlast 的值为 1，发送接口状态机仍留在

BMD_TX_DMA_RST 状态。

(2)若此时发送的是存储器写请求 TLP，则信号 $s_axis_tx_vaild=1$ 、信号 $dma_wib_ren=1$ 和信号 $wbuf_ren=1$ ，且 PCIE 发送存储器写请求 TLP 的头标和数据负载分别由信号 wib_dout 和信号 $wbuf$ 的数据组成，其中，根据 $wbuf$ 信号的数据组成的 TLP 为数据负载的存储器写请求 TLP 可知此时要发送的是存储器写请求 TLP，发送数据时 BMD_TX_DMA_RST 状态需跳转至 BMD_TX_DMA_Mwr_QW 状态，在跳转后 BMD_TX_DMA_Mwr_QW 状态下发送未发送完成的存储器写请求 TLP。

(3)若此时发送的是存储器读完成 TLP， $dma_req_rd_addr = dma_req_addr$ 且 $dma_reg_rd_en=1$ 。因在该状态下可防止寄存器堆产生延迟，为因发送 TLP 产生的延迟需在图 4.6 的 B 条件下跳转到 BMD_TX_DMA_CPLD 并完成 TLP 的组建与发送。

BMD_TX_DMA_MWR_QW 状态：当发送接口状态机进入此状态时，表示发送引擎正在进行存储器写请求 TLP，在该状态下，信号 $s_axis_tx_vaild=1$ 和信号 $wbuf_ren=1$ ，由于发送引擎进行存储器写请求 TLP 时不用读取新的 TLP 头标，则信号 $dma_wib_ren=0$ 。与此同时使用信号 $wbuf_dout$ 的数据构成 PCIE 进行传输时存储器写请求 TLP 的数据负载，此外使用信号 $wtlp_length_cnt$ 对已发送的数据进行计数，判断信号 $wtlp_length_cnt$ 值是否达到 TLP 的大小。

(1)若达到，当前存储器写请求 TLP 操作完成，发送引擎将使信号 $s_axis_tx_tlast=1$ 、信号 $wtlp_complete=1$ ，状态跳转到 BMD_TX_DMA_RST 状态。

(2)若未达到，信号 $wtlp_complete=0$ ，状态继续停留在 BMD_TX_DMA_MWR_QW。

BMD_TX_DMA_CPLD 状态：当发送接口状态机进入此状态，表示发送引擎模块中正在执行存储器读完成 TLP。在 BMD_TX_DMA_CPLD 状态下，信号 $s_axis_tx_tvaild=1$ 以及信号 $s_axis_tx_tlast=1$ ，同时将接收引擎模块接收存储器

读请求 TLP 头标中的信息和 dma_req_rd_data 的数据来组成 PCIE 传输时存储器读完成 TLP 的头标和数据负载，存储器读完成 TLP 的数据发送完成后需进行状态的跳转，即在 BMD_TX_DMA_CPLD 状态跳转至 BMD_TX_DMA_RST 状态。

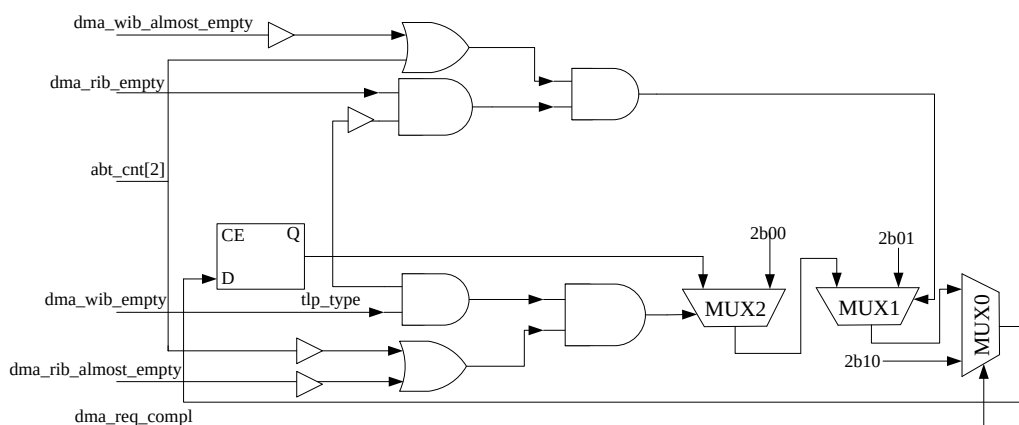


图 4.7 事务仲裁逻辑结构图

事务类型仲裁逻辑的主要作用为发送引擎模块根据 2.2.2 节的 TLP 类型选择一个 TLP 进行数据传输。发送引擎模块中的事务类型使用 Verilog HDL 硬件语言对仲裁逻辑进行 RTL 设计，并给出了事务类型仲裁逻辑的关系。在本设计中的 TLP 类型包括以下三种：(1) 存储器读请求 TLP；(2) 存储器写请求 TLP；(3) 存储器读完成 TLP。其中优先级最高的 TLP 类型为(1)，则优先调用(1)，另外两个类型将使用优先级轮询的方式进行仲裁，(1)和(3)的优先级分别设置为以下两个状态：状态 A 和状态 B，并将这两个状态的关系表示为 A 和 B，状态 A：写>读；状态 B：读>写。事务型仲裁逻辑选择仲裁计数器的数值决定优先级状态的切换，计数值以 4 为界限进行状态 A 和状态 B 的选择。

如图 4.7 所示为事务类型仲裁器的逻辑实现过程，其中包括 MUX0、MUX1 和 MUX2，三个 MUX 实现具有优先级的多级条件判断。下面介绍在三个 MUX 条件下的逻辑实现。在 MUX0 条件下，由 dma_req_compl 信号选择其输出，当 dma_req_compl=1，将 2'b10 = tlp_type，此时应发送存储器读完成 TLP。当 dma_req_compl=0，则进入 MUX1 条件下，MUX1 的输出由逻辑表达式 $A=A1\&\&A2\&\&(A3\|A4)$ ，其中 $A1=\neg Tlp_type[0]$ ， $A2=dma_rib_empty$ ， $A3=abt_cnt[2]$ ， $A4=\neg dma_wib_almost_empty$ ，若 A=1 时，将 2'b01=tlp_type，此时应发送存取器写

请求 TLP，若 $A=0$ 时，将转入 MUX2 的条件，这时 MUX2 的值由 $B=B1\&\&B2\&\&(B3\|B4)$ 决定，其中 $B1=!tlp_type[0]$ ， $B2=dma_wib_empty$ ， $B3=!abt_cnt[2]$ ， $B4=!dma_rib_almost_empty$ ，若 $B=1$ ， $2'b00=tlp_type$ ，此时应发送存储器读请求 TLP，若 $B=0$ ，则 tlp_type 的值不变。

4.1.4 接收引擎模块

根据 4.1.1 节对 PCIE IP 核协议进行解释，在 4.1.3 节设计发送引擎的时候了解本文的数据不仅包括端点设备和主机间的 DMA 方式进行传输，还包括读写访问存储空间的 PIO 模式，所以接收引擎模块的工作过程为：

1、对于以 DMA 方式进行的数据传输，接收引擎模块通过 TRN TO AXI-S 从 PCIE IP 核接收带数据的存储器读完成 TLP，对存储器读完成 TLP 解析后从中提取出相关信息，并根据这些信息传输给 DDR3 存储器进行数据存储。

2、对于以 PIO 方式进行的数据传输，接收引擎直接通过 TRN TO AXI-S 从 PCIE IP 核接收来自主机的配置数据。

如图 4.8 所示，接收引擎模块的逻辑设计由接收接口状态机和 DMA 接口转换模块两部分组成。

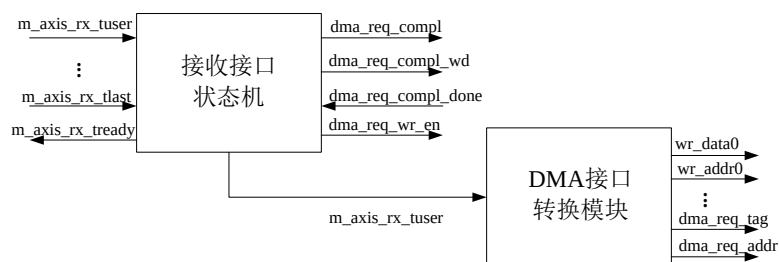


图 4.8 接收引擎模块结构图

接收引擎模块直接连接到 PCIE IP 核的用户接口，通过 AXI-S 接口进行数据的接收，PCIE IP 核与接收引擎模块发送 TLP 是通过 AXI-S 接口进行传输的，AXI-S 接口的 $m_axis_rx_tuser$ 、 $m_axis_rx_tvalid$ 和 $m_axis_rx_tlast$ 等 m_axis 信号接入接收引擎，其中接收引擎的接收接口状态机以控制数据流的传递。接收接口状态机和 DMA 接口转换模块用信号 $m_axis_rx_tuser[127:0]$ 进行连接，接收接口状态机利

用 dma_req_compl、dma_req_compl_wd 和 dma_req_compl_done 等 completion TLP 信号来实现逻辑控制；DMA 接口转换模块使用以下三组数据信号：RX_DMA_Mwr_Group、RX_DMA_Mrd_Group 和 RX_DMA_CPLD_Group 进行控制。

表 4.3 接收引擎的接口信号及定义

端口名	输入\出	描述
tx_tdata	Input	AXI-S的m_axis发送端口
tx_tkeep	Input	
tx_tlast	Input	
tx_tuser	Input	
tx_tvalid	Input	
tx_tready	Output	
dma_req_compl	Output	TLP完成包的端口
dma_req_compl_wd	Output	
dma_compl_done	Output	
dma_req_tc	Output	向发送引擎传送完成TLP所需信息
dma_req_td	Output	
dma_req_ep	Output	
dma_req_len	Output	
dma_req_tag	Output	
dma_req_addr	Output	TLP写寄存器端口
mem_reg_wr_en	Output	
mem_reg_wr_addr	Output	

下面在介绍接收引擎模块状态机之前，先用表 4.3 接收引擎模块的接口信号及定义进行说明。

接收引擎模块通过如图 4.9 所示的状态机实现接收接口的逻辑功能，状态机的三个状态分别为 BMD_RX_IDLE 状态、BMD_RX_PIO_WAIT 状态和 BMD_RX_CPLD_QW 状态。这三个状态的定义及转移过程如下：

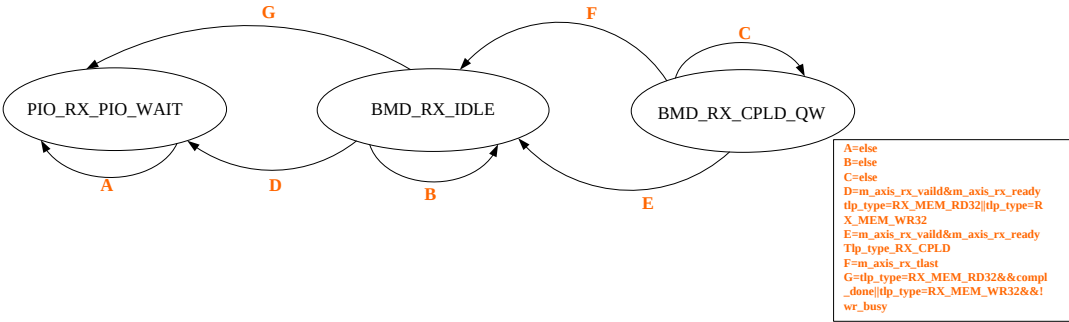


图 4.9 接收接口的状态转移图

BMD_RX_IDLE 状态: 接收接口状态机的初始状态, 在此状态下对接收到的 TLP 类型进行判断, 根据不同 TLP 的类型跳转至不同状态。当接收引擎接收到下一个 TLP 时, 信号 $m_axis_rx_tvaild$ 、 $m_axis_rx_tready=1$, 这时 AXI-S 的发送接口端口的信号 rx_tdata 出现 TLP 包含 TLP header 和 Data Payload 的 4DW。TLP 头标的格式和内容都会随着 TLP 类型和路由方式的改变而改变, TLP 类型的不同会使数据处理不同, 同时也会使接收引擎之间的状态转移发生变化。以下对 3 种 TLP 类型进行说明:

(1)当发送的 TLP 类型为存储器写请求 TLP, 即对 TLP 头标的 4DW 进行处理, 从而解析出写地址和写数据, 并由接收引擎根据 TLP 类型发生状态变化可知接收接口状态机的 BMD_RX_IDLE 状态需要跳转至 BMD_RX_PIO_WAIT 状态。

(2)当发送的 TLP 类型为存储器读请求 TLP, 将对 TLP 头标的 4DW 进行处理从而解析出读地址, 并解析出 TLP 头标 4DW 中的 Requester ID、TC 和 EP 等字段, 完成解析后发送至 4.1.3 节的发送引擎模块产生存储器读完成 TLP, 此时接收引擎根据 TLP 类型的发生状态变化可知接收接口状态机的 BMD_RX_IDLE 状态需要跳转至 BMD_RX_PIO_WAIT 状态。

(3)当发送的 TLP 类型为存储器读完成 TLP, 对 TLP 头标的 4DW 进行处理, 从而解析出 Tag、Byte Count 和 Length 字段, 并由接收引擎根据 TLP 类型发生状态变化可知接收接口状态机的 BMD_RX_IDLE 状态需要跳转至 BMD_RX_CPLD_QW 状态。

BMD_RX_PIO_WAIT 状态: 该状态为数据传输时 PIO 操作的等待状态, 信号 $m_axis_rx_tready=0$, PCIE IP 核停止接收数据。TLP 的类型决定数据传输的过程, 以下对 2 种 TLP 类型进行说明:

(1)当发送的 TLP 类型为存储器写请求 TLP, 则向 4.1.2 节的寄存器堆模块中写入数据, 在寄存器写入过程中, 信号 $wr_busy=0$, 这时触发 BMD_RX_PIO_WAIT 状态跳转至 BMD_RX_IDLE 的条件为信号 $wr_busy=1$, 信号 wr_busy 位于上升沿后处于 1 时, 表明数据全部成功写入寄存器堆模块中。

(2)当发送的 TLP 类型为存储器读请求 TLP, 首先从存储器读请求 TLP 头标中解析出 TD 等字段, 然后将解析出的字段发送给 4.1.3 节的发送引擎模块, 最后由

发送引擎模块根据解析出的字段等信息组成的存储器读完成 TLP 并将其发出，此时当数据传输时的信号 `compl_done = 1` 时，表明发送引擎模块发送完成，`BMD_RX_PIO_WAIT` 状态跳转至 `BMD_RX_IDLE` 状态。

BMD_RX_CPLD_QW 状态：该状态在数据传输时的接收引擎模块继续接收存储器读完成 TLP，持续向 Tag 缓存区中写入数据，此时信号 `tag_buf_wen=1`。当信号 `m_axis_rx_tlast=1` 时表明存储器读完成 TLP 的数据已经全部接收，`BMD_RX_CPLD_QW` 状态由此跳转至 `BMD_RX_IDLE` 状态，条件跳转是在 `m_axis_rx_tvalid&m_axis_rx_tready`和`tlp_type=RX_CPLD`下进行转换。

如表 4.4 所示为 DMA 接口转换模块状态转移过程中产生的信号定义表，分别用 A、B、C 表示状态转移过程中产生的三组信号，A 为 `RX_DMA_Mwr_Group`、B 为 `RX_DMA_Mrd_Group`，C 为 `RX_DMA_CPLD_Group`，根据 `RX_TLP_Paeser` 控制信号的值决定 DMA 接口转换模块的 A、B、C，若 `RX_TLP_Paeser` 的值为 0 时对应 DMA 接口转换模块三组中的 A；`RX_TLP_Paeser` 的值为 1 时对应 DMA 接口转换模块三组中的 B；`RX_TLP_Paeser` 为 2 时对应 DMA 接口转换模块三组中的 C。

表 4.4 DMA 接口转换模块的信号定义表

端口号	输入\出	描述
<code>buf_wdata</code>	Output	<code>RX_CPLD_Group</code> 组的Tag写数据
<code>wr_datan(n=0\1)</code>	Output	<code>RX_Mwr_Group</code> 组的寄存器写地址和写数据
<code>wr_addrn(n=0\1)</code>	Output	
<code>dma_req_tc</code>	Output	<code>RX_Mrd_Group</code> 组的存储器读TLP的头标和数据
<code>dma_req_td</code>	Output	
<code>dma_req_ep</code>	Output	
<code>dma_req_len</code>	Output	
<code>dma_req_attr</code>	Output	
<code>dma_req_tag</code>	Output	
<code>dma_req_addr</code>	Output	

4.1.5 中断引擎模块

PCIE 通信支持 MSI 中断，为了达到最高传输效率，本文的数据传输通过中断方式进行，为保证中断处理的正确性，本文选用 MSI 中断。中断引擎的主要作用是在 DMA 读/写操作过程中出现中断时做出控制处理，主要功能是通过 4.2.2 节 DMA 引擎来负责 8 通道中数据传输时的 DMA 状态进而在数据传输过程发出中断请求。即在本设计完成一次 DMA 传输后，中断引擎向 4.1.1 节中的 PCIE IP 核发出 MSI 中断请求，因 PCIE IP 核内部的中断机制可满足本设计 MSI 中断需求，则本设计收到中断请求时将会通过 PCIE IP 核内部的中断机制向上位机发出中断消息。

下面在介绍中断引擎模块状态机之前，先用表 4.5 中断引擎模块的接口信号及定义进行说明，表中 dma_done_chn(n=0~7) 为本设计中 8 通道 dma_done 的端口信号，信号 cfg_interrupt_n 负责向 IP 核发送 MSI 中断请求，信号 cfg_interrupt_rdy_n=1 时说明 PCIE IP 核成功发送了 MSI 中断请求。

表 4.5 中断引擎的接口信号及定义

端口名	输入\出	描述
dma_done_chn(n/0~7)	Input	通道的数据传输已完成
msi_int_release	Input	MSI中断的信号
cfg_interrupt_n	Output	向IP核发出MSI中断
cfg_interrupt_rdy_n	Output	发送MSI成功的信号值为1

如图 4.10 所示为中断引擎状态机的状态转移过程，转移过程中的四个状态分别为 MSI_IDLE 状态、MSI_SEND 状态、MSI_SENT 状态和 MSI_LOCK 状态。这四个状态的转移关系及定义如下：

MSI_IDLE 状态：该状态为初始状态，在该状态下当 8 通道中的任一通道 DMA 传输完成，信号 wchn_dma_done=1 和信号 rchn_dma_done=1，MSI_IDLE 状态跳转至中断引擎状态机的 MSI_SEND 状态，否则在 MSI_IDLE 状态下继续等待。

MSI_SEND 状态：在该状态下 PCIE IP 核发出 MSI 中断请求后由 MSI_SEND 状态跳转至 MSI_SENT 状态。

MSI_SENT 状态：在该状态下，MSI 中断消息在 PCIE IP 核内部生成，当 PCIE IP 核成功发送给计算机内存 MSI 中断消息后，状态由 MSI_SENT 状态转移到 MSI_INT_LOCK 状态，此时信号 `cfg_interrupt_rdy_n` 处于 1。

MSI_LOCK 状态：在中断引擎状态转移过程中一直处于 MSI_LOCK 状态，直到上位机端成功响应中断后状态随之发生变化，且在响应 MSI 中断前不再发出新的中断请求。此时 MSI_LOCK 状态跳转至 MSI_IDLE 状态的条件是 PC 响应中断，则逻辑信号 `int_release` 由 0 变为 1。

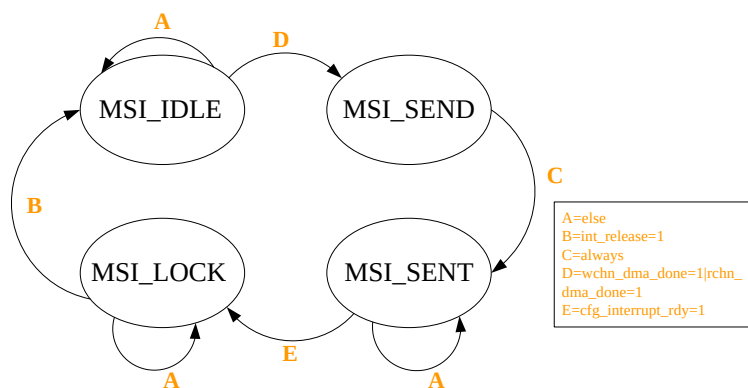


图 4.10 中断引擎的状态转移图

4.2 PCIE DMA 控制逻辑的设计与实现

PCIE DMA 控制逻辑在数据传输通道的职责是保证数据能正确的被接收和发送。本设计在上一节的基础上已实现了高速数据传输通道中的接口逻辑，下面设计并实现数据传输通道的 PCIE DMA 控制逻辑。PCIE DMA 控制逻辑由 DDR3 存储器、DMA 引擎和通道仲裁器组成。

4.2.1 DDR3 存储器

DDR3 存储器与 DMA 引擎和通道仲裁器连接，并控制着数据传输通道时的 DMA 状态，因 DDR3 存储器在上传数据和下传数据时的读写控制逻辑一样，本节通过上传数据通道对 DDR3 存储器进行设计。

DDR3 存储器负责将数据写入到 DDR3 内存中或者从 DDR3 内存中读取，对

DDR3 进行读写控制，从而实现数据的缓存。DDR3 存储器将数据缓存到 DDR3 中，因本文设计的是 8 个数据通道，则需设置 8 个 DDR3 存储空间，用第一内存、第二内存……第 n 内存 n 表示，如图 4.11 所示。其中 DDR3 存储器模块控制 DDR3 内存的读写，故把它封装成类 FIFO 接口，用第一 FIFO、第二 FIFO……第 nFIFO 表示，并根据存储地址把 DDR3 存储器模块分成不同区域进行不同类型的数据缓存。



图 4.11 DDR3 缓存空间结构图

本文 DDR3 存储器使用突发访问方式进行读写操作，DDR3 存储器由 FIFO 接口、通道仲裁和 MIG IP 核三部分组成，结构如图 4.12 所示。由图可知，FIFO 接口为方便 FPGA 逻辑设计对 DDR3 存储器进行访问，将 DDR3 存储器的接口对应于图 4.11 的缓存空间封装成类 FIFO0~FIFO7 接口；通道仲裁连接着 FIFO 接口和 MIG IP 核，其功能是当多个通道的申请读写请求时进行仲裁处理，其中 DDR3 存储器的 MIG IP 对外接口包括信号线、数据线以及控制线，但因 DDR3 提供一组读写总线，所以 DDR3 在数据传输的相同时间里只能支持 8 通道中的任一个通道进行读、写操作，对于本设计来说当多个通道访问时，必须要实时回应，所以需对通道的读、写操作进行仲裁；MIG IP 核由 Xilinx 7 系列 FPGA 自带的 IP 硬核，可对 DDR3 进行读写访问。通过对 DDR3 存储器的设计可使 DDR3 更加适合多通道高速数据传输系统，便于 DDR3 作为整个高速数据传输时的缓存，达到了大容量缓存的效果。本文主要对通道仲裁的设计进行详细说明。

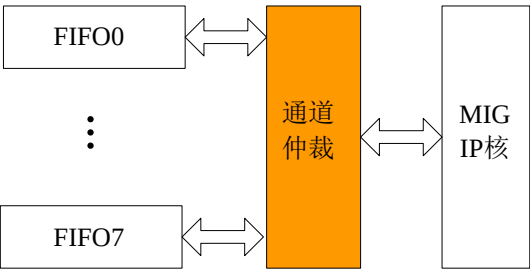


图 4.12 DDR3 存储器结构图

通道仲裁模块主要负责多个通道访问 DDR3 控制器时的总线切换，采用轮询仲裁确定下一通道，通道发送的 TLP 指令通过轮询方式发送到 DMA 引擎模块，进而提高数据的处理效率。DDR3 存储器读写速度快，但读和写不能同时进行，本设计使用 DDR3 通道仲裁进行 IP 核接口的转换，避免读写冲突问题。

为了更清楚的说明通道仲裁模块的设计方案，先介绍 Xilinx 公司的 Kintex-7 芯片提供的 DDR3 生成控制 IP 核的 MIG 相关接口，由图可知本文将 MIG 的接口分为命令通道接口、写数据通道接口和读数据通道接口，表 4.6 所示为 MIG 的主要端口。写通道信号只能进行数据写操作，由 ddr_addr、ddr_cmd 和 ddr_en 等信号管脚控制；读通道信号只能进行数据读操作，由 ddr_wdata、ddr_wr_en 和 ddr_wdata_rdy 等信号管脚控制，命令通道信号同时参与数据的读写操作，由 ddr_rdata 和 ddr_rdata_valid 等信号管脚控制。

表 4.6 MIG 的接口信号及定义

端口名	描述	
ddr_addr	命令通道接口	请求地址由MIG IP核决定
ddr_cmd		请求命令由MIG IP核决定
ddr_en		使能信号
ddr_rdy		MIG IP核在数据传输时接收命令
ddr_wdata	写通道接口	向DDR3写数据
ddr_wr_en		使能信号
ddr_wdata_rdy		MIG IP核在数据传输将接收写数据
ddr_rdata	读通道接口	从MIG IP核读出数据
ddr_rdata_valid		有效信号

通道仲裁模块对于这三类通道接口对应有三个形式的 FIFO。三类通道接口添加 FIFO 外围电路设计框图如图 4.13 所示，读数据缓冲为 rdata_fifo_ctrl，读命令

的缓冲为 `rcmd_fifo_ctrl`，写数据缓冲 FIFO 和写命令缓冲 FIFO 分别为 `wdata_fifo_ctrl` 和 `wcmd_fifo_ctrl`，`arb` 为本设计仲裁器，其中写数据 FIFO 用于缓存、同步 PCIE 写操作数据；同时读数据 FIFO 用于缓存、同步待返回的 PCIE 读操作数据。

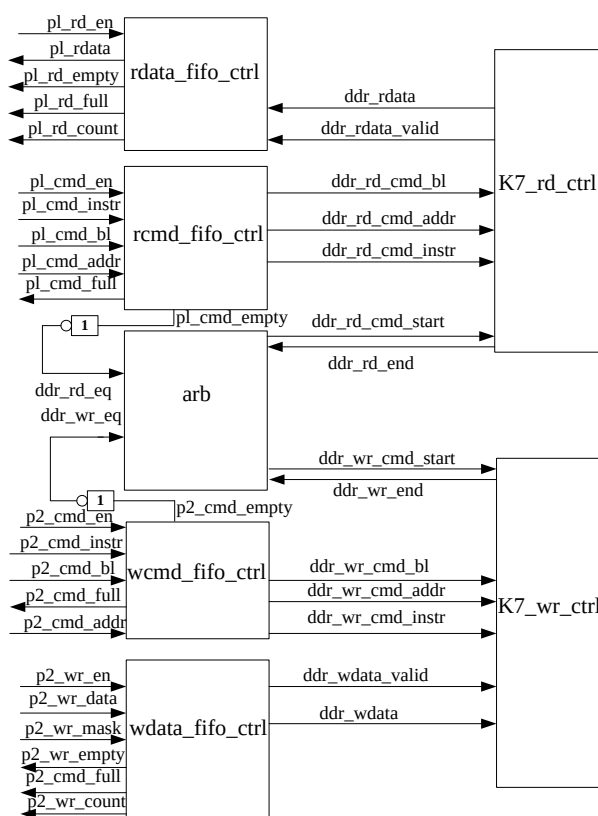


图 4.13 添加 FIFO 建构图

本设计对没有及时执行的命令和数据缓存起来，在本次操作结束后，再从 DMA 引擎命令缓冲区内取下一次需要执行的命令和数据，命令类 FIFO 和读数据类 FIFO 和写数据类 FIFO 对执行的命令和数据缓存起来，并且配合 3 类 FIFO 的读写控制逻辑，表 4.6 中构建成的三类 FIFO 接口信号需转换为 MIG IP 核接口信号来完成数据缓冲。下面介绍命令 FIFO 和写数据 FIFO 以及读数据 FIFO 的模块设计图。

如图 4.14 所示为 `cmd_FIFO` 模块的结构框图，由 A、B、C、D 四部分组成，A 为突发长度计数器、B 为通道计数器、C 为 MUX、D 为 `cmd_FIFO` 读写控制逻辑。A 要完成发送通道中读/写数据的个数进行统计，当通道中统计的个数超过最大突

发长度时通知 B 并选择要传输的通道；B 负责通道间的切换，因上传数据和下传数据共需 0-15 个数据通道，轮询规则将从 0 到 15 的通道数依次进行，通道的读或写使能信号中的任何一个有效都会对 DDR3 进行访问；C 由 0~15 个通道组成，其中 0~7 为通道的读地址和读数据，8~15 为通道的写数据和写命令，通道计数器负责对 0~15 的通道进行切换；D 负责对 DDR3 中的命令 FIFO 进行读写控制，1、若 cmd_FIFO 为写使能信号，在 8 通道中选择下一次进行传输的通道，2、若 cmd_FIFO 为读使能信号，则信号 cmd_FIFO 非空、信号 ddr_rdy 变为有效，当信号发生变化后将 cmd_FIFO 模块中缓存的命令和地址传输至 MIG IP 中。

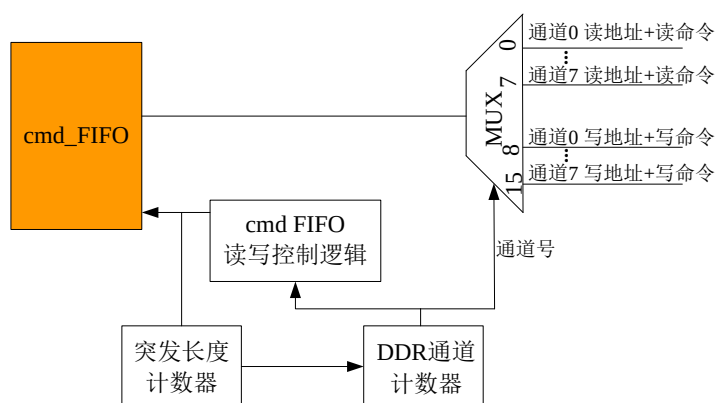


图 4.14 cmd_FIFO 模块结构图

wdata_FIFO 模块是在 cmd_FIFO 模块基础上进行设计，保持原有的通道计数器和突发长度计数器，改变了 MUX 和 cmd_FIFO 读写控制逻辑，因 wdata_FIFO 只对写数据进行处理，则由图可知 MUX 只需对 8-15 通道数进行响应。wdata_FIFO 读写控制逻辑与 cmd_FIFO 读写控制逻辑不同，负责对写数据 FIFO 进行读写控制。

(1)当信号 ddr_cmd=000，且信号 ddr_en 有效，wdata_FIFO 为写使能信号。(2)当信号 ddr_wdata_rdy 有效，wdata_FIFO 为读使能信号，此时 wdata_FIFO 非空。

wdata_FIFO 模块结构图如图 4.15 所示。



图 4.15 写数据 FIFO 的模块结构图

rchn_FIFO 与图 4.14 和图 4.15 两个 FIFO 模块结构不同，如图 4.16 所示。

rchn_FIFO 保存读操作的通道数，不保存数据，rchn_FIFO 读写控制逻辑与 cmd_FIFO 读写控制逻辑和 wdata_FIFO 读写控制逻辑不同，负责对 rchn_FIFO 进行读写控制。(1)如果此时要存储数据传输的通道号，则读数据 rchn_FIFO 为写使能时存储的通道号即为当前 rchn_FIFO 存入的通道号，且信号 ddr_cmd=001、信号 ddr_en 信号有效。(2)如果此时要读取数据传输的通道号，则读数据 rchn_FIFO 为读使能时读取的通道号即为当前 ddr_rdata 的通道号，且信号 ddr_rdata_valid 有效。选择器起到选择作用，rchn_FIFO 读写控制逻辑首先根据条件(1)或条件(2)将数据传输至 rchn_FIFO 后，然后再由 rchn_FIFO 输出的通道号，最后对接入选择器的 rdata 以及 rdata_valid 进行处理。

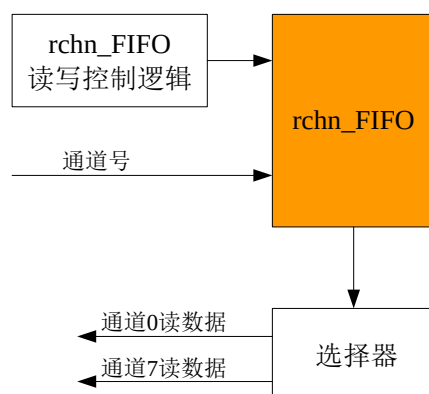


图 4.16 读数据 FIFO 的模块结构图

4.2.2 DMA 引擎

DMA 引擎分别完成 DMA 写操作和 DMA 读操作，其对应着上传数据通道和下载数据通道。

1、当 DMA 引擎执行写操作时，DMA 引擎将 DDR3 缓存区中的数据搬移到发送引擎模块，DMA 引擎首先收到 4.2.3 节通道仲裁器发出写数据/信息的使能信号和 8 通道中将要传输的通道号后按照该通道的 DMA 传输长度和传输起始地址一起组建寄存器写请求 TLP 头标，然后将其存储在数据 FIFO 和命令 FIFO 中，并将使能信号和通道号从相应通道的数据缓存区中读取后存储在数据 FIFO 和命令 FIFO 中，最后发送引擎模块取出 TLP 数据载荷和 TLP 头标后生成 TLP 并发送，其生成 TLP 的过程是由 DMA 引擎的数据 FIFO 和命令 FIFO 的状态决定的。

2、当 DMA 引擎执行读操作时，DMA 引擎首先根据 4.2.3 节通道仲裁器发出写数据/信息的使能信号和 8 通道中将要传输的通道号，然后在此基础上根据 DMA 传输时的长度和起始地址组成存储器读请求 TLP 的头标，最后通过组建的存储器读请求 TLP 将数据存储在 DMA 引擎的数据 FIFO 和命令 FIFO 中。

为存储 DMA 引擎的数据进入 FIFO 中时的运行地址不超过 DMA 缓冲区的存储空间，则本设计采用地址掩码寄存器完成 DMA 环形缓冲区。如图 4.17 所示为利用地址掩码寄存器产生环形缓冲区的逻辑结构图，源/目的地址可设置为 A、B、C 和 D，其中 A=0、B=1、C=-1、D=索引，修正后进入 MUX。

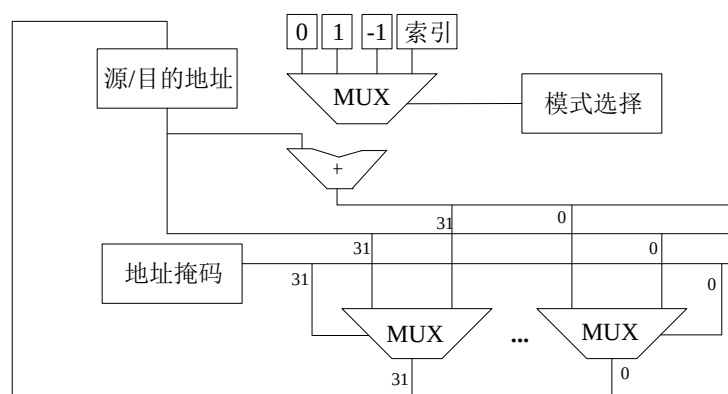


图 4.17 利用地址掩码寄存器产生环形缓冲区

图 4.18 是采用命令缓冲机制后的 DMA 引擎传输示意图，在 FPGA 端增加了命令存储区。DMA 引擎由命令 FIFO 和数据 FIFO 组成，其中命令 FIFO 包括读命令 FIFO 和写命令 FIFO。如图 4.19 所示为 DMA 引擎请求读/写命令 FIFO 格式

的结构图，由图可知，地址 ID 占 6b、FLAG 占 1b，其中 FLAG 用于判断 DMA 引擎写命令或 DMA 引擎读命令，写命令 FIFO 对应上传的数据通道，读命令 FIFO 对应下传的数据通道，但两者模块设计的实现方式是一样。

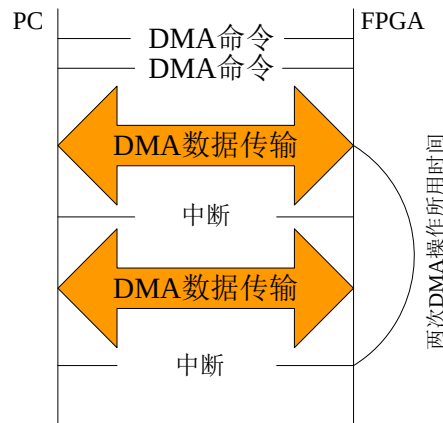


图 4.18 命令缓冲区下 DMA 传输流程

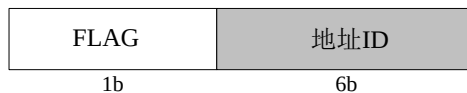


图 4.19 DMA 引擎的请求命令格式

本文对写命令 FIFO 和 MW_r TLP 数据 FIFO 的实现方式进行详细介绍。DMA 引擎的接口信号及其定义如表 4.7 所示。在 MW_r TLP 数据 FIFO 中 DMA 引擎只负责对 FIFO 进行写数据，读数据由 PCIE DMA 接口设计的发送引擎完成。

表 4.7 DMA 引擎的接口信号及定义

端口名	输入\出	描述
dma_wib_ren	Input	数据传输写信息时的读ren信号
dma_wib_dout	Output	数据传输写信息时的读dout信号
dma_wdb_ren	Output	数据传输写数据时的读ren信号
dma_wdb_dout	Input	数据传输写数据时的读dout信号
wchn_data_chn(n/0~7)	Input	数据传输时 写数据缓存区间的数据信号/读使能信号/有效信号
wchn_ren	Output	
wchn_data_valid	Input	
wdma_burst_en	Input	数据进行写传输时 一个突发长度有效信号/通道号/结束信号
wdma_burst_chn	Input	
wdma_burst_last	Output	
wdma_len_chn(n/0~7)	Input	8通道进行数据传输时的写DMA 起始地址/写DMA传输长度
tran_wr_addr	Input	
dma_wib_empty	Output	写信息FIFO是否为空
dma_wib_almost_empty	Output	

命令 FIFO 模块设计图如图 4.20 所示。主要由 A、B、C 这三部分组成，其中 A 为 wib_FIFO 写使能信号产生逻辑、B 为 TLP 传输地址产生逻辑、C 为 Burst 传输 TLP 计数器。wib_FIFO 的写数据由 TLP 传输地址以及 wdma_burst_chn 和 mwr_tlp_size 动态拼接而成，wib_FIFO 的多个命令传输完成后，利用动态拼接策略。C 对命令 FIFO 的 Mwr TLP 头标进行计数统计，当该计数器中的统计值达到最大突发长度时，在使能 B 输出的信号后通知 4.2.3 节的通道仲裁器模块进行通道切换。wib_FIFO 写使能信号产生逻辑的数据除了通过信号 wib_FIFO_wen 发送到 wib_FIFO，还发送到图 4.20 中的 TLP 传输地址产生逻辑和图 4.20 中的 Burst 传输 TLP 计数器。wib_FIFO 右边的信号为发送接口模块控制的 FIFO 读相关信号。

下面对命令 FIFO 模块中 A 和 B 的设计进行详细说明。

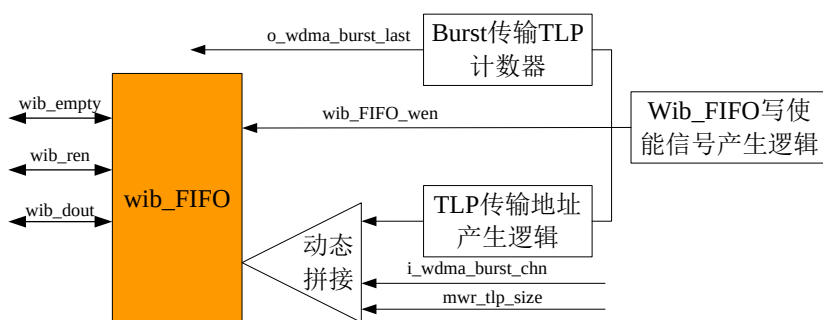


图 4.20 命令 FIFO 的模块结构图

A 结构框图主要由 A1、B1、C1 三部分组成，A1 为寄存器、B1 为 MUX，A1 和 B1 组成一个计数器 C1，C1 为 mwr_tlp_data_cont，如图 4.21 所示。如图 4.21 的橘色框内的计数器的作用是为了计数 TLP 数据负载，发送数据时在命令 FIFO 缓冲区中每读出一个数据在计数器初始值 mwr_tlp_size 的基础上就会减 1。当 TLP 数据负载传输完成，计数器 mwr_tlp_data_cont 的计数值变为 0，当 burst_tlp_cnt>0 时，则此时计数器的计数值需返回计数器的初始值 mwr_tlp_size，此时 mwr_tlp_data_cont 计数器开始计数下一个 TLP 的数据负载。当计数器的计数值为 1、信号 wdma_burst_en=1 和信号 wib_FIFO_full=0，表示此时只剩下一个数据，当前 TLP 传输完成后信号 wib_FIFO_wen 变为有效。

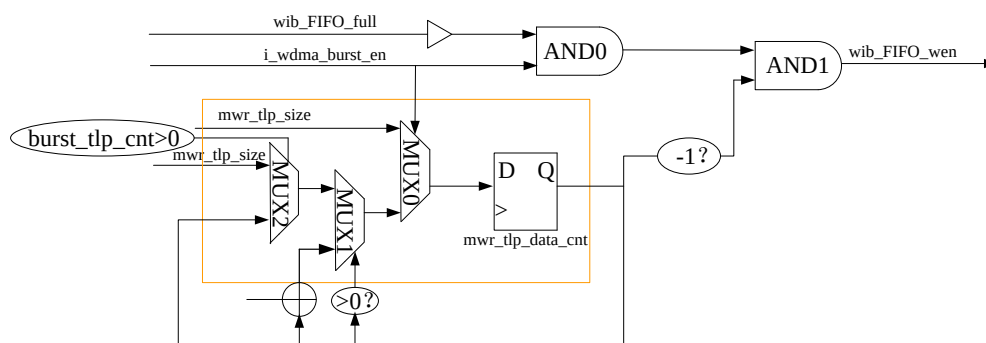


图 4.21 wib_FIFO 写使能信号产生逻辑结构图

B 结构主要由 A2、B2、C2 组成，A2 为寄存器、B2 为 MUX0、C2 为 MUX1，如图 4.22 所示。逻辑单元 POS_DET 的功能是检测电路，当逻辑单元输出为 1 时，此时信号 dma_en 的值为 1，这时 8 通道中的某一通道开始传输 DMA 数据，并且寄存器被赋值为 8 通道中将要传输通道的 mwr_tlp_init_addr 信号值。在进行 DMA 传输时每生成一个 TLP 头标，DMA 写信息 FIFO 的读使能 wib_en 信号就变为有效，当 wib_en 有效时将成为下一个 TLP 传输数据的目的地址，这时的寄存器会在当前的基础上加一个 TLP 的长度。

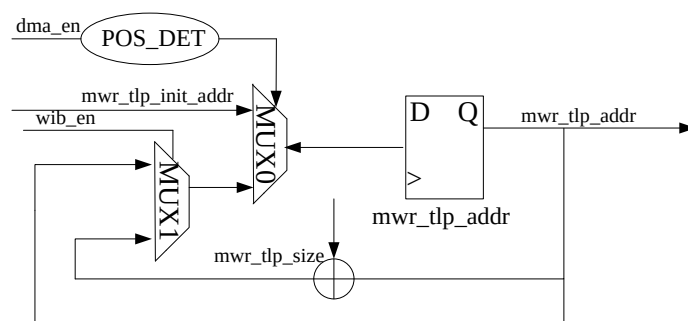


图 4.22 TLP 传输地址产生逻辑结构图

wdb_FIFO 逻辑结构图主要由 A、B、C 组成，A 为多路复用器、B 为选择器、C 为 wdb_FIFO，如图 4.23 所示。当 wdb_FIFO 写使能信号有效，信号 i_wdma_burst_en=1，A 提供 FIFO 写数据，信号 wdb_FIFO_full=0，此时要传输的通道号为 i_wdma_burst_chn(n=0~7)，A 选择的数据授权通道号由授权通道号选择

后发送至 wdb_FIFO。B 提供 8 通道中数据缓存区的读使能信号，由授权号 i_wdma_burst_chn 决定并输出 wchn_ren_chn (n 为通道数) 的信号值。FIFO 左边的信号同样是由发送引擎模块控制关于 FIFO 读操作的相关信号。

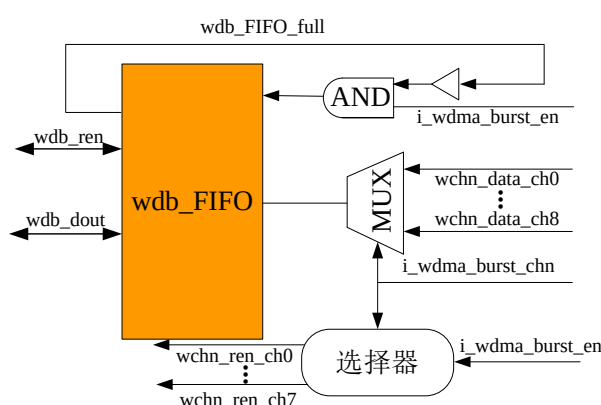


图 4.23 数据 FIFO 的模块结构图

4.2.3 通道仲裁器

通道仲裁器接收 DMA 引擎的配置，在多通道同时发起 DMA 请求时，进行通道仲裁，DMA 组包后的数据通过仲裁模块选择其中一个通道的通道号和传输使能信号进行上传，并根据 8 通道信号带宽的优先级选择下一次数据传输的通道号进行传输。

通道仲裁器模块的通道仲裁逻辑指示哪个通道被仲裁，以此为各个通道进行优先级的选择，根据优先级提出仲裁请求，进入通道仲裁器，然后经过仲裁批准，进行通道选择。因通道仲裁器在上传数据和下传数据时的控制逻辑一致，本节通过上传数据通道对通道仲裁器进行设计，本设计根据控制逻辑从 8 通道中选择对应的通道进行数据传输，从而实现 FPGA 与计算机之间的数据传输。

通道仲裁器根据本设计需求将通道分为 4 个优先级组，4 个优先级组分别为 I~IV，且 I~IV 优先级组中的任一组都包括 2 个通道，此时信号带宽相同的通道则分到 I~IV 优先级组中的同一组，同一组包含 2 个通道中的优先级与信号带宽成正相关的关系，在数据传输时优先级组内进行通道仲裁时采用的是轮询方式。

通道仲裁器的优先级表中的数据设置如下：1、将 I 组从通道 0 到通道 7 的值

设置为 1、0、1、0、0、1、0、0。2、将 II 组从通道 0 到通道 7 的值设置为 0、1、0、1、0、0、0、0。3、将 III 组从通道 0 到通道 7 的值设置为 0、0、0、0、1、0、0、1。4、将 IV 组从通道 0 到通道 7 的值设置为 0、0、0、0、0、0、1、0。上面 1、2、3、4 中的每个数据都代表一个单元格。

通道仲裁器的逻辑结构图由 A、B、C、D 组成，A 为 ARB_ST、B 为 CH_BW_Upd、C 为 PRI_BUILD、D 为 CH_ARB，如图 4.24 所示。A 对其他电路逻辑状态进行控制，并产生使能信号 o_tran_en、CH_ARB_en 和 Grp_Bld_en；B 监测寄存器模块中信号带宽，根据信号带宽产生 CH_BW_Upd[n](n=0~7)信号后对其进行更新操作；C 根据各通道的信号带宽做出优先级的选择；D 根据优先级及通道的状态选择出相应的通道进行通道仲裁，并通过 o_ch_n 信号选择下次的传输通道号。下面对通道仲裁器模块中 A、B、C 和 D 的设计进行详细说明。

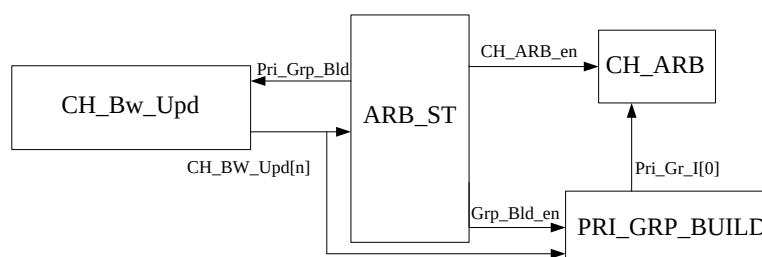


图 4.24 通道仲裁器的逻辑结构

如表 4.8 为通道仲裁器的端口信号及含义。

表 4.8 通道仲裁器的接口信号及定义

端口名	输入/出	描述
clk	Output	工作时钟
rst	Output	复位信号
ch_bw	Input	各通道信号带宽
ch_bw_vaild	Output	各通道信号带宽的有效信号
ch_rq	Output	各通道传输的请求信号
ch_n	Input	各通道中下次传输的通道号
tran_en	Output	DMA传输的使能信号
tran_over	Input	标志DMA传输的完成信号

如图 4.25 所示为 A 的状态转移图，ARB_ST 由三个状态组成，分别为

PRI_GRP_BUILD 状态、PRI_ARB 状态和 DTA_TRAN 状态。各个状态的含义及转移过程如下:

PRI_GRP_BUILD 状态: 在此状态下系统复位进行优先级表的构建。该状态为 ARB_ST 的初始状态, 当信号 CH_BW_Upd[n](n=0~7) 中的一个信号或多个信号等 1 时, ARB_ST 状态机中的使能信号 Grp_Build_en=1, 此时相应通道的信号带宽发生变化, 通知 PRI_GRP_BUILD 进行优先级表的更新, 在优先级更新的过程中, 当 Pri_Grp_Building=1 时, 说明上面相应通道中的信号带宽已同步 I~IV 组的优先级表中, PRI_GRP_BUILD 状态转移到 PRI_ARB 状态。

PRI_ARB 状态: 在此状态下使能信号 CH_ARB_en=1 时对传输通道进行仲裁处理。在进行数据传输时的通道仲裁有以下两种情况: 若有通道申请传输, 此时 PRI_ARB 状态转到 DTA_TRAN 状态, 下次进行数据传输的通道由通道仲裁器构建的优先级表中信息决定; 若没有通道传输, 信号带宽发生变化, PRI_ARB 状态转到 PRI_GRP_BUILD 状态, 若没有发生变化, 则状态保持不变。

DTA_TRAN 状态: 该状态下实现数据传输的功能。进入 DTA_TRAN 状态后, 使能信号 o_tran_en=1 时通知 4.2.2 节的 DMA 引擎开始传输数据。当数据传输完成一次数据传输任务后, 使能信号 i_tran_over=1。在传输过程中如果监测到信号 CH_BW_Upd[n](n=0~7) 中任一信号值等于 1, 即监测到某一通道信号带宽发生改变, 这时在传输数据完成后, DTA_TRAN 状态跳转至 PRI_GRP_BUILD 状态, 否则在数据传输完成后 DTA_TRAN 状态跳转至 PRI_ARB 状态。数据的传输任务未完成将会一直留在 DTA_TRAN 状态。

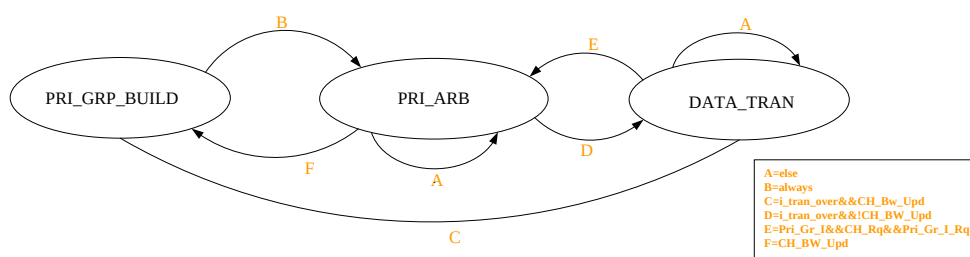


图 4.25 ARB_ST 的状态转移图

B 存在于 8 个通道内，为进行 8 通道的数据传输，本设计有 8 个逻辑单元，且每个逻辑单元的结构都一样，则本设计以通道 0 的逻辑单元结构为例进行说明。CH_BW_Upd 逻辑结构由 A2、B2 组成，A2 为 CH_Bw_Upd[0]、B2 为 MUX，如图 4.26 所示。其中存储带宽更新指示信号为图 4.26 中的 CH_Bw_Upd[0]，三个 MUX 为 CH_BW_Upd 结构中的条件判断，MUX0、MUX1 构成 CH_BW_Upd 结构中第二级条件判断，MUX2 为 CH_BW_Upd 结构中第一级的条件判断，且在第一级判断时寄存器的值由 MUX2 的地址信号决定。除了下面 1、2 两种情况，信号 CH_Bw_Upd[n](n=0) 的值保持不变。1、在数据传输进行下一时钟周期时信号 CH_Bw_Upd[n](n=0) 的值保持不变。1、在数据传输进行下一时钟周期时信号 CH_Bw_Upd[n](n=0) 的值变为 1，此时信号 CH_Bw_Upd[n](n=0) 的值为 0，同时信号 CH_Bw_vaid[n](n=0) 的值也为 0。2、在数据传输进行下一时钟周期时信号 CH_Bw_vaid[n](n=0) 的值也为 0。2、在数据传输进行下一时钟周期时信号 CH_Bw_Upd[n](n=0) 为 0，信号 CH_Bw_Upd[n](n=0) 为 1，同时 Pri_Grp_Building 信号值也为 1。

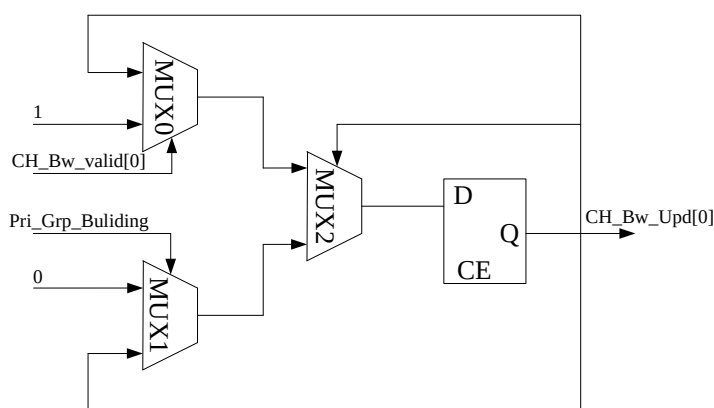


图 4.26 CH_Bw_Upd 结构图

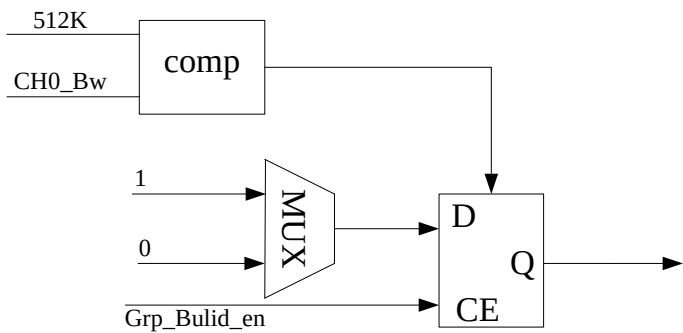


图 4.27 PRI_BUILD 结构图

C 为通道仲裁器构建优先级表, 优先级表构建过程中的每个数字表示存放着优先级构建逻辑单元, 即有 $4 \times 8 = 32$ 个数据的单元格, 因每个数据中单元格的优先级构建逻辑结构一样, 所以对 I~IV 中的通道 0 来描述。优先级组构建逻辑单元主要由 A3、B3、C3 组成, A3 为 comp、B3 为 MUX、C3 为 Pri_Gr_I[0], 如图 4.27 所示。当通道 0 的 CH0_Bw 带宽为 512K, Grp_Build_en=1 时, Pri_Gr_I[0] 输出 1, 则将对对应优先级逻辑结构的单元格设置为 1, 否则为 0。

如图 4.28 所示, D 包括 4 个 A4、4 个 B4、1 个 C4, A4 为轮循单元、B4 为 MUX、C4 为寄存器。其中轮循单元 0~3 负责 I~IV 通道组的内部轮循, 当 I~IV 中的其中一个通道组被授权, 则轮循单元 0~3 根据其内部轮循顺序决定下次的传输通道。4 个 MUX 负责 4 个通道组之间的仲裁, MUX0~MUX3 根据通道组 pri_Gr_I_Rq~pri_Gr_IV_Rq 的优先级条件来判断电路。下面对 A4 轮循单元的设计进行详细说明。

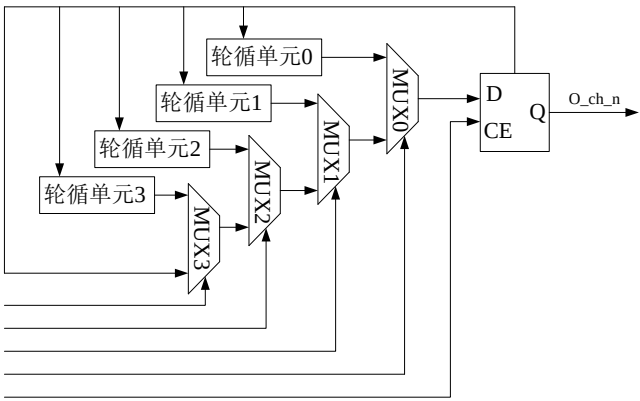


图 4.28 CH_ARB 结构图

如图 4.28 所示的 4 个轮循单元, 4 个轮询, 0 到 3 绝对优先级, 本文仅对轮循

单元 0 的电路逻辑进行介绍，因图 4.28 中的轮循单元 0~轮循单元 3 内部的电路逻辑一样，如图 4.29 所示为轮循单元 0 的结构。图中寄存器输出信号为 chn_Q ，输入信号为 chn_D ，则当 $chn_D=chn_Q$ 时，比较器输出为 1，否则为 0。因本设计需对数据传输时的通道数进行计数，则本设计轮循单元的计数器由图 4.29 中的寄存器、MUX0 和比较器组成，计数器从 0-8 开始循环计数，计数到的数即为通道数，若计数器的计数值为 0，即通道 0 开始进行传输，此时 $CH_Rq[n](n=0)=1$ 和 $Pri_Gr_I[n](n=0)=1$ ，计数器的值被输出，输出值将作为下次传输授权的通道号，其他情况下输出信号 chn_Q ，这时获得传输的通道号保持不变。

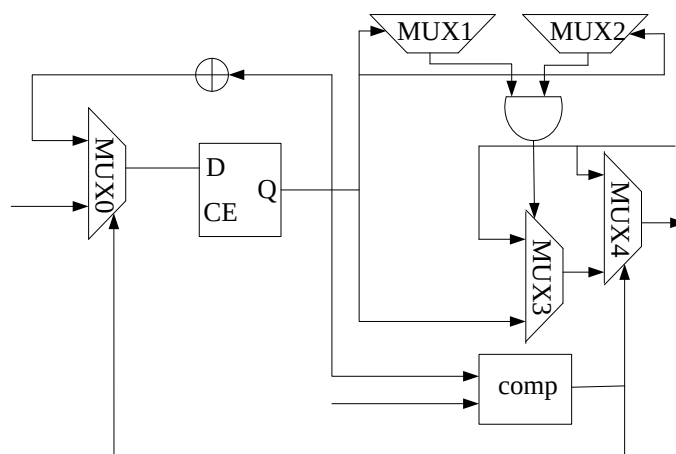


图 4.29 轮循单元结构图

4.3 本章小结

本章为数据传输通道的 FPGA 设计章节，对各个模块进行详细设计，其中本文基于 FPGA 实现的所有设计中，PCIE IP 核、接收引擎、发送引擎、中断引擎和寄存器堆的 PCIE 接口逻辑设计达到了高速数据传输的要求，PCIE DMA 控制逻辑的 DDR3 存储器、DMA 引擎和通道仲裁器实现了本设计多通道传输的需求。

第五章 实验验证与性能分析

本文第四章对数据传输通道的各模块进行了设计，本章为验证设计的正确性和合理性，对设计的数据传输通道进行仿真验证与性能分析。本设计测试所用的软件平台为 ISE14.7、Matlab 和 Modelsim se，操作系统为 Windows 10，即首先对数据传输通道的平台进行介绍，然后在 ISE14.7 完成编译与综合，在 Modelsim se 和 Matlab 环境下进行功能验证，最后对数据传输通道进行性能测试与分析。

5.1 数据传输通道的测试平台

基于 PCIE 总线的高速数据传输通道的软件测试平台包括上位机软件和 ISE 开发平台所提供的测试环境，接下来对这两部分进行介绍。

(1) 上位机软件

基于 PCIE 总线的高速数据传输通道的上位机软件包括驱动程序和应用程序接口^[57]，如图 5.1 所示为上位机软件组成结构。驱动程序是运行在特定硬件目标上的软件程序文件，它充当固件和主机操作系统之间的接口，应用程序接口作为函数接口的一种，为用户提供一系列数据控制。

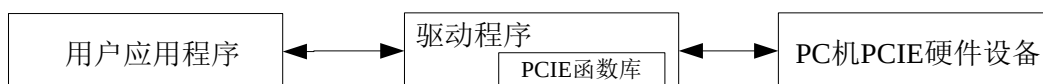


图 5.1 上位机软件组成结构

驱动是硬件与软件的桥梁，本设计数据传输通道的交互是在驱动程序的作用下由硬件和软件一起控制的^[58]。本设计的开发基于 PCIE 的 WinDriver 驱动程序，Jungo 公司推出了应用于图 5.1 所示的上位机软件组成结构中的驱动开发工具 WinDriver，在本设计中为达到只需进行顶层应用程序的开发即可完成上位机与 FPGA 的设计，本文对驱动程序中 PCIE 的函数库使用 WinDiver 进行开发，从而降低了本设计中 PCIE 驱动的开发时间和成本。本设计用 WinDriver 开发 PCIE 驱动的程序结构如图 5.2 所示。

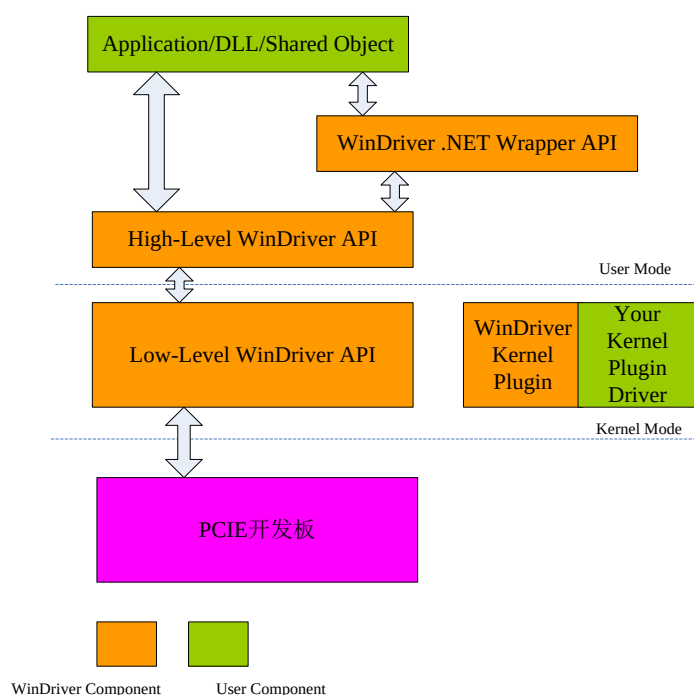


图 5.2 基于 WinDriver 的驱动程序结构

通过驱动程序完成底层初始化后，进入数据传输的相关功能函数。本文利用 WinDriver 的 API 函数在上位机上实现具有 GUI 的应用程序^[59]。该用户应用程序的步骤如下：

初始化 PCIE 设备后，再对 DMA Buffer 空间进行初始化，然后启动 DMA 回环测试，并设定并启动外部数据 PCIE 读/写，最后在上位机中进行数据的写入和存储。

（2）ISE 开发平台

ISE 作为硬件开发平台下 Xilinx 公司 K7 系列推出的 FPGA 集成开发环境，集成了多种开发和测试环境^[60]，其中数据传输通道的 FPGA 逻辑设计的开发和测试便是在 ISE 的开发平台上实现，包括利用 Verilog HDL 硬件语言对 FPGA 逻辑设计进行输入、综合、布局布线和 bit 流文件的生成。最后通过 ISE 联合 Modelsim se 进行功能仿真，从而分析数据传输通道的设计是否达到要求。

以上为数据传输通道的软件测试平台，即在 ISE14.7 上进行 FPGA 逻辑设计的开发后，通过 PCIE 总线传输到被测 MK7325FA 开发板的主机上，进而实现 PC 机与 FPGA 板卡之间的数据传输。

5.2 数据传输通道各模块的仿真验证

本文第四章为数据传输通道的 FPGA 设计, 本节对 FPGA 逻辑设计的 DDR3 存储器模块、DMA 引擎模块和通道仲裁器模块进行仿真验证, 以验证本设计的正确性。

5.2.1 DDR3 存储器模块的仿真验证

本文为验证 8 通道同时访问一块 DDR3 产生冲突问题时的存储和读取数据的正确性, 需对 DDR3 模块在 ISE 调用 Modelsim 执行仿真验证, 其 DDR3 模块的仿真波形图如图 5.3 所示。

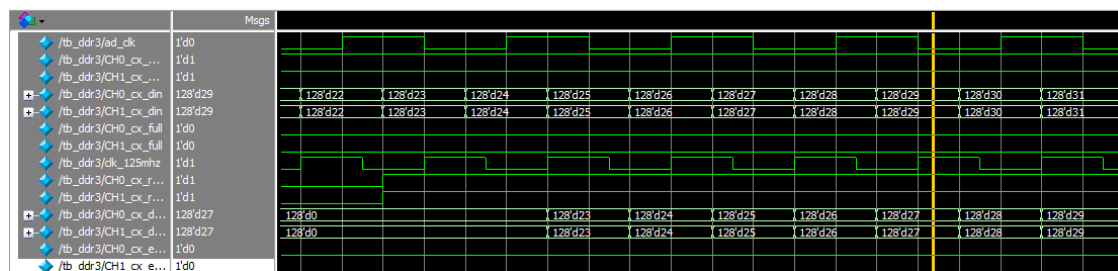


图 5.3 DDR3 存储器仿真图

篇幅有限, 本文只展示了通道 0 和通道 1 同时访问 DDR3 存储器时的读写仿真波形图, 可以看出, 当 CH0_empty 和 CH1_empty 信号值同为 0, CH0_wr_en 和 CH1_wr_en 信号值同为 1 时, 两通道可以同时进行写数据并且写入数据的值相同; 当 CH0_empty 和 CH1_empty 信号值同为 0, 且 CH0_rd_en 和 CH1_rd_en 信号值同为 1 时, 两通道可以同时进行读数据并且读取数据的值相同。由图可知, 通道 0 和通道 1 可以同时访问 DDR3 存储器, 进行读写操作, 并且当通道 0 和通道 1 访问 DDR3 存储器时, 两通道的写入数据能正确读取到对应的数据线上, 证明本设计的 DDR3 寄存器能正确进行写入和读取操作。同样, 当 8 通道同时访问 DDR3 存储器, 也可以同时进行读写操作。

5.2.2 DMA 引擎模块的仿真验证

本文为验证 DMA 引擎模块是否符合本设计的要求, 对上传数据和下传数据再 ISE 调用 Modelsim 进行仿真验证, 图 5.4、图 5.5 分别显示的是 DMA 上传数据和

下传数据的仿真图。

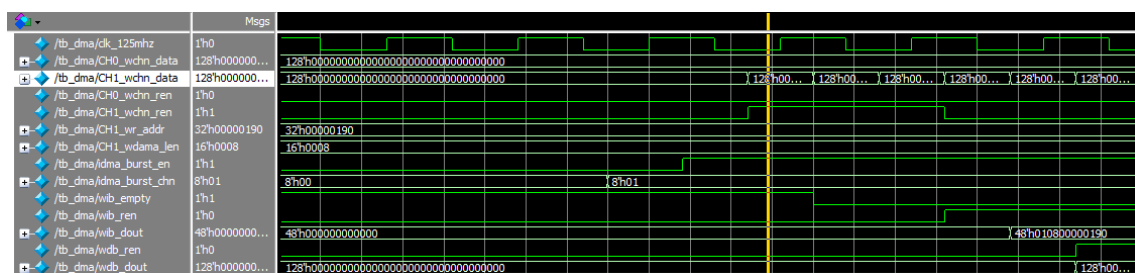


图 5.4 DMA 上传数据仿真图

如图 5.4 所示, idma_burst_chn 信号值由 0 变为 1, 同时 idma_burst_en 信号值也由 0 变为 1 后, DMA 引擎会使 CH1_wchn_ren 信号值变为 1, 进行数据的读取操作。在 DMA 引擎进行读取操作时将生成的 TLP 头标存储在写命令 FIFO 中, 写命令 FIFO 的 wib_empty 由 1 变为 0, 为读取 TLP 头标和数据负载, 发送引擎模块的 wib_ren 信号值变为 1。由图可知, 写命令 FIFO 的读数据 wib_dout 的信号值与上传数据时 DMA 引擎的信息一致, 同时, 写数据 FIFO 的 wdb_dout[127:0]信号值与 CH1_wchn_data[127:0]信号值也一致。表明 DMA 引擎在上传数据时写命令 FIFO 和写数据 FIFO 的功能符合本设计要求。

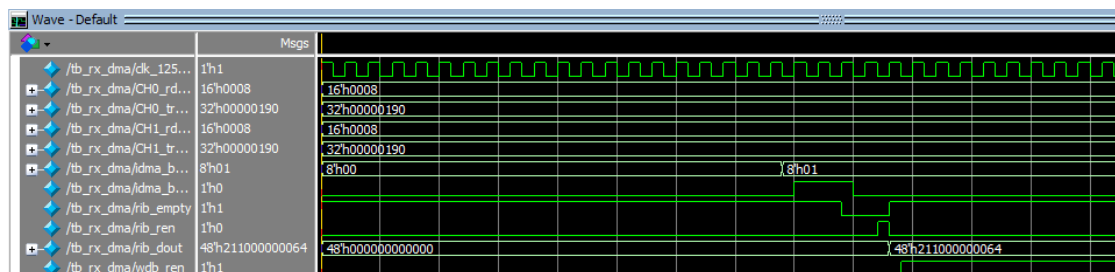


图 5.5 DMA 下传数据仿真图

如图 5.5 所示, DMA 引擎的读使能信号有效时, DMA 引擎首先根据数据传输时的 TLP 类型生成相对应的 TLP 头标, 然后存储在 DMA 引擎相应的 FIFO 中, 此时, rib_ren 信号值由 0 变为 1。由图可知 rib_dout[47:0]的值与下传数据时 DMA 引擎的信息相同, 说明 DMA 引擎下传数据时读命令 FIFO 的读数据功能符合本设计要求。

由图 5.4 和图 5.5 可知该设计能完成 DMA 读写功能并保证数据的正确传输。

5.2.3 通道仲裁器模块的仿真验证

因每个通道并没有明确优先级顺序，为保证 8 通道能有效地获准当前需要进行传输通道号，首先对各通道进行配置，将通道 0 和 1 的带宽设置成 1024KB/s，同时将通道 3 和 4 的带宽设置成 512KB/s；然后根据 4.2.3 节设计的通道仲裁器进行通道的优先级选择，根据优先级的选择知道此次传输的通道号为 0、1、2、3；最后用理论上传输的通道号与 ISE 调用 Modelsim 的仿真验证结果进行对比。

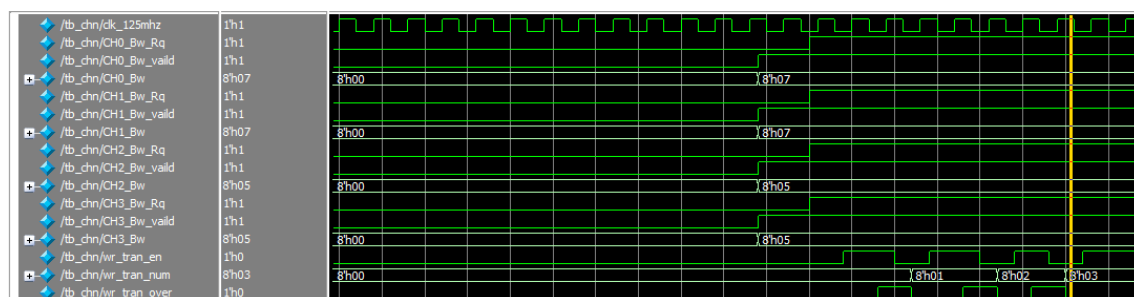


图 5.6 通道仲裁器仿真图

如图 5.6 所示在通道仲裁器的仿真图中没有收到访问通道消息时，处于 IDLE 状态，`o_wr_tran_num[7:0]` 的值为 0，`CH[n]_Bw_vaild` 的值由 0 变为 1，带宽更新；`CH[n]_Bw_Rq` 的值由 0 变为 1，通道 0、1、2 和 3 开始申请传输，根据 4 个通道带宽更新后的优先级大小选择要传输的通道，此时 `o_wr_tran_en` 的值由 0 变为 1；在一个周期后，`i_wr_tran_over` 的值由 1 变为 0，DMA 传输结束，此时获得优先级高的通道号进行传输，这时 `o_wr_tran_num[7:0]` 的值为 01，01 即为此时传输的通道号，每当一次 DMA 传输完成时，`o_wr_tran_num[7:0]` 的值即变为 02、03、04，软件测试的结果和理论结果一致，达到了设计目标。

5.3 数据传输通道整体的仿真验证

5.2 节分别对数据传输通道设计中的 DDR3 存储器模块、DMA 引擎模块以及通道仲裁器模块在 ISE14.7 调用 Modelsim se 进行了仿真验证，证明了数据传输通道的各模块的功能无误，达到了本文设计要求。本设计要完成一次传输，需经过 DDR3 存储器、接收引擎、发送引擎和 DMA 引擎等一系列过程，为验证数据传输的正确性，调用 Modelsim 对数据传输通道进行仿真验证，得到如图 5.7 所示的仿

真图。

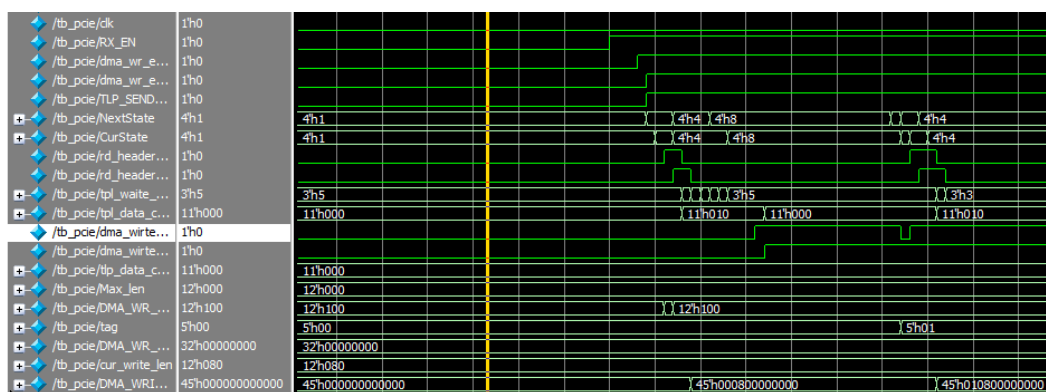


图 5.7 数据传输通道仿真图

由 5.7 所示的仿真图可知,数据传输的时钟周期为 4ns,数据包大小为 256Bytes,进行一次数据传输时有效数据的时钟周期为 30 个,根据一次传输的时钟周期和数据包的大小可以大约算出 PCIE 进行数据传输时的带宽为 22Gbps,由此可以看出本设计数据传输通道达到了设计要求。

下面将对 3.1 节的多通道无线接收机传输模型在图 5.8 的平台上执行整体的仿真测试,以验证该数据传输通道整体设计的正确性。其中验证平台的四个通道依次通过如下流程:激励源—DDC—数据传输通道—计算机,每个通道对数字中频信号进行模拟后产生位宽为 16bit 的 102.4Mbps 的数据。其中,数字中频信号的 40MHz 通过 DDC 模块变为零频后,再通过数字下变频的信号带宽做一系列的滤波处理,然后生成 IQ 数据;数据传输通道将 IQ 数据发送到计算机中进行信号的处理。

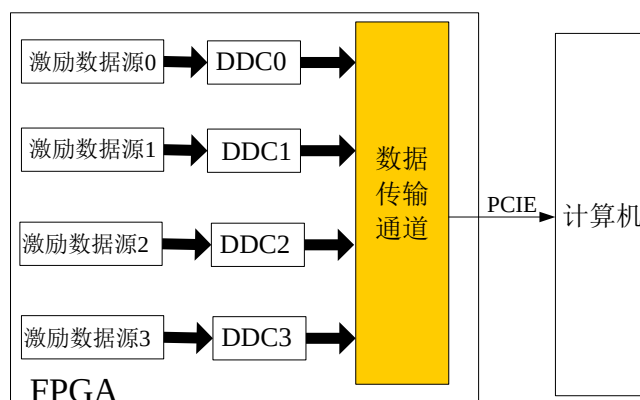


图 5.8 数据传输通道的平台

测试过程为:模拟信号经 ADC 采样后形成数字信号,由激励数据源对 ADC 接收到的数字信号进行模拟;然后,DDC 模块实现数字下变频的处理,并输出两

路中频信号 $I(n)$ 和 $Q(n)$ ，生成基带 IQ 数据，最后通过数据传输通道将 IQ 数据发送到计算机中进行信号的处理。

此次测试，将 DDC 可选的带宽与 IQ 数据的传输速率分别设置为 400KHz 和 1MB/s。同样在计算机上将各个通道的信号带宽设置为 400KHz，设置完信号带宽后启动数据传输。

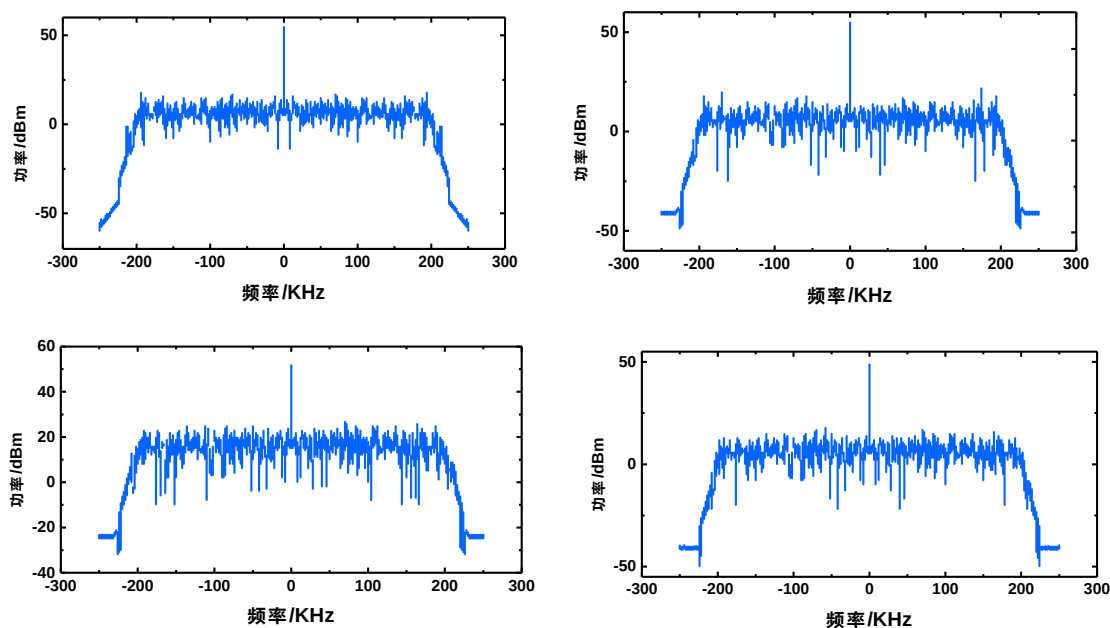


图 5.9 各通道带宽为 400KHz 时，IQ 数据频谱图

计算机接收到各通道的基带 IQ 数据后通过 Matlab 对其画出频谱图，如图 5.9 所示。由图可知，依次为通道 0、1、2、3 的频谱图，各通道信号峰值频率为 0，带宽为 400KHz，与计算机设置的信号带宽一致，说明本设计的数据传输通道能将数据正确的发送到计算机上。

接下来，为了更直观的验证数据传输通道传输数据的正确性，本文在计算机运行状态时，向 FPGA 中的 DDC 模块发送命令，同时为模拟实际环境中的带宽改变，将各通道的信号带宽设置为如表 5.1 所示。

表 5.1 各通道信号带宽表

信号带宽	400KHz	300KHz	200KHz	150KHz
通道号	0	1	2	3

计算机接收到各通道的数据后通过 Matlab 对其画出频谱图，如图 5.10 所示。

图为通道 0、1、2、3 改变带宽后的频谱图，经测试，各通道信号带宽和表 5.1 中的信号带宽一样，说明 DDC 模块在接收到计算机下发的信号带宽后能将各通道的数据正确传输。

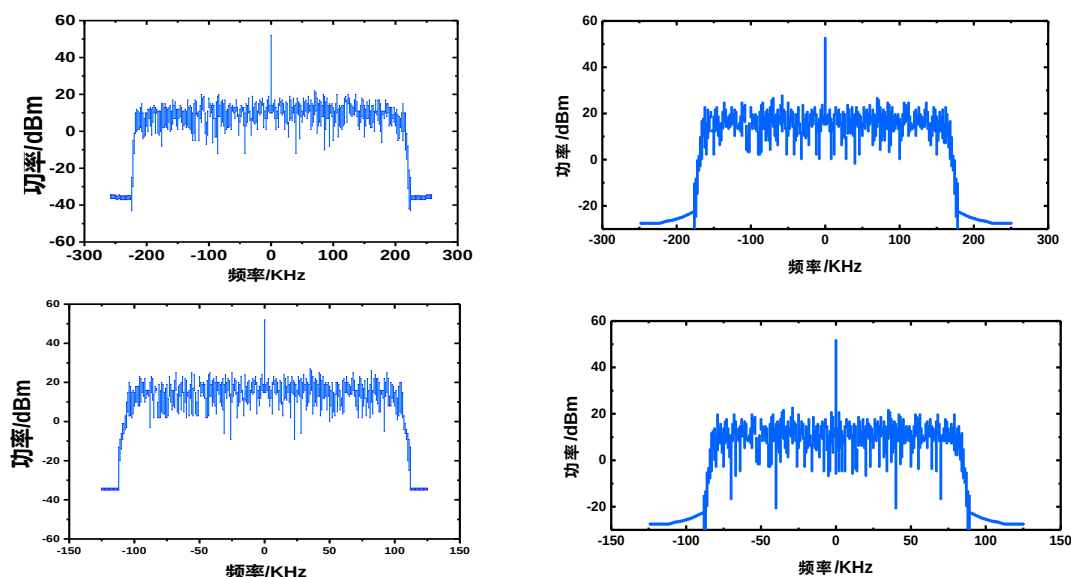


图 5.10 各通道带宽改变后，IQ 数据频谱图

5.4 数据传输通道的性能测试

在 5.2 节对 PCIE 数据传输的功能仿真无误下，本节对 PCIE 传输进行以下 3 项测试。此节性能测试的目的是为了验证第四章的设计是否达到本设计数据传输速率的要求，在测试过程中，为进一步测试数据传输速率的准确性，在 FPGA 逻辑设计中设置一款 DMA 传输时间的计数器。

1、通道数为 8 不变，改变激励数据单通道速率

本设计在 5.1 节的软硬件测试平台上采用 FPGA—PC—FPGA 数据流进行回环测试，数据传输时的回环测试过程如下：首先由计算机发出开始指令，然后 FPGA 把 PCIE DMA 控制逻辑中的激励数据读出，最后数据通过 PCIE 总线将传输到计算机中。启动传输功能回环测试的流程图如图 5.11 所示。

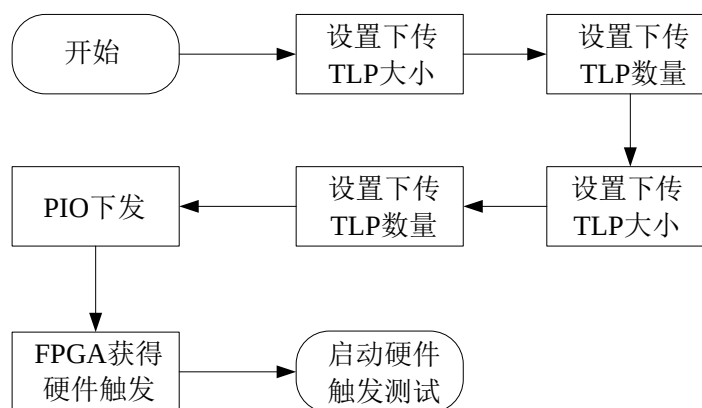


图 5.11 回环测试流程图

表 5.2 通道数为 8，改变激励数

数据激励单通道速率 (MB/s)	上传数据速率 (MB/s)	下传数据速率 (MB/s)
1	7.92	7.76
10	79.86	78.62
100	792.5	791.3
200	1582	1542
300	2362	2310
320	2420	2382

本文按照数据传输通道的 FPGA 逻辑设计，将其通过 ISE14.7 综合、映射与布局布线，生成 bit 文件下载到 Xilinx K7 开发板中，并把 Xilinx K7 开发板插到带有 PCIE 插槽主板的计算机中。在软件程序接口处设置 DMA 长度和各通道的数据源传输速率后，启动回环测试。如表 5.2 所示的结果是改变激励数据传输速率得到的，由表 5.2 中的数据可知，在通道数为 8 不变的情况下，改变激励数据单通道的速率，上传数据和下传数据的速率小于 8 倍的单通道速率，但其 8 通道速率的 FPGA—PC 速度的峰值为 2420MB/s，PC—FPGA 速度的峰值为 2382MB/s，均取得了较显著的实验效果。

2、通道数为 8 不变，改变 DMA 传输数据量大小

为测试不同 DMA 传输数据量大小与数据传输速率大小的关系，在通道数不变的情况下，本文结合计数时钟 250MHz 算出 DMA 读写不同数据量时的速率，如图 5.12 所示。图 5.12 所示为通道数不变，改变 DMA 传输数据量大小，DMA 数据传输速率的测试结果，由图可知数据传输速率随着 DMA 传输长度的增加而增加，当传输数据从 1KB 增加到 32KB 时，数据传输速率的值呈直线式增长，DMA 传输的

数据量达到某个特定值时，将缓慢呈现平行于横坐标趋势。

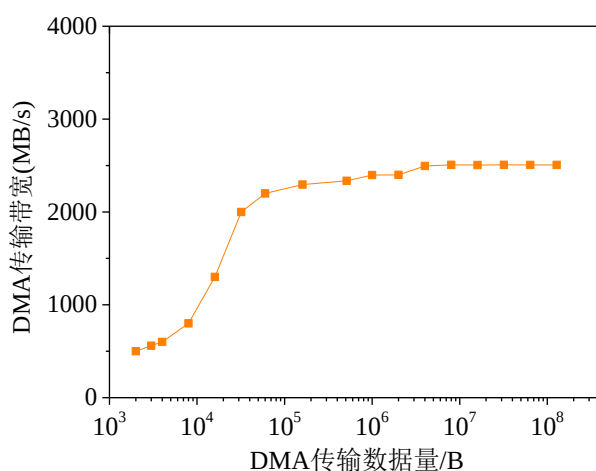


图 5.12 DMA 数据传输速率的变化曲线

3、DMA 传输长度不变，改变通道数的大小

为验证不同通道数与数据传输带宽的关系，在 DMA 传输长度不变的情况下改变通道号进行综合，生成不同通道中 FPGA Xilinx 开发板需要的 bit 文件。如图 5.13 所示为不同通道数对应的数据传送带宽的测试结果，随着通道数的增多，总的传输速率逐渐减低。由图可知，当 DMA 长度一定时，单通道的 DMA 传输带宽最高即图中红色线，8 通道的 DMA 传输带宽最低即图中蓝色线，因 DMA 长度一定，每个通道共享 DMA 长度，所以横坐标中 8 通道的 DMA 长度最短，即出现这一现象的原因是通道数越多，通道仲裁器所消耗的延迟就越多，DMA 传输带宽也就越来越低。

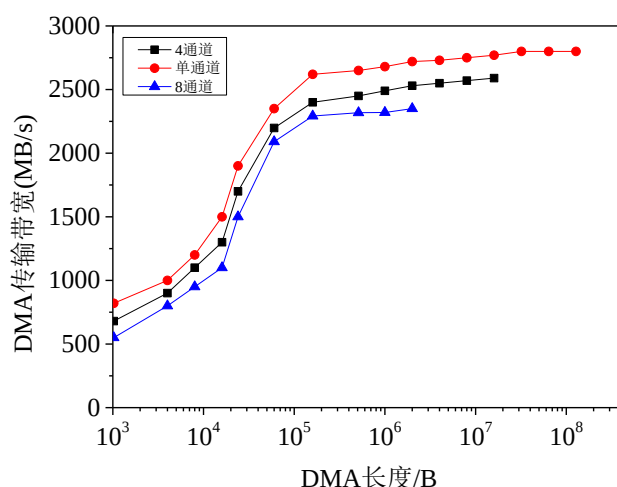


图 5.13 多个通道数对应的数据传输带宽测试结果图

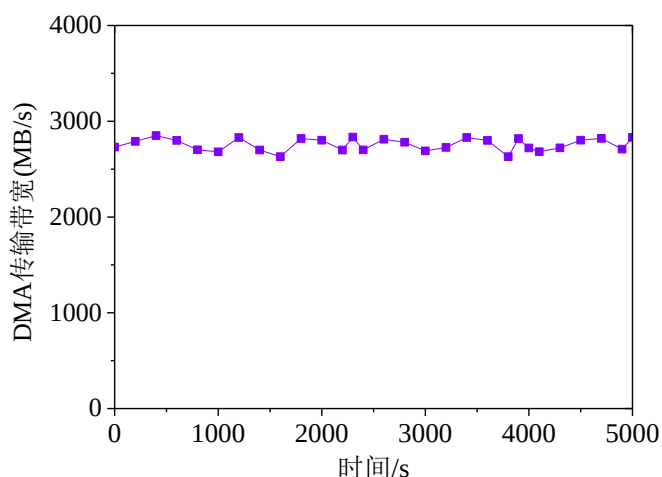


图 5.14 PCIE Gen2 单通道下稳定性测试结果

根据上面 (3) 的性能测试结果可知, 本设计可以达到比较高的传输带宽, 即数据传输通道在单通道时和 8 通道时的最大传输带宽为 2830MB/s 和 2325MB/s, 之后在 PCIE Gen2.0 x8 的模式下对数据传输通道在单通道时稳定性进行测试, 如图 5.14 所示。经测试, PCIE 数据传输性能稳定在 2830MB/s, 达到本设计的预期要求。综上, 数据速率有大幅度的提高, 能满足各种高速数据传输。

由文献[29]的公式可知在相同工作模式下, 比较数据传输通道在单通道的传输速度即能得到在 8 通道传输性能的优劣。本设计的数据传输通道在单通道时与文献[35]和文献[34]相比, 数据传输速率提高约 300MB/s, 图 5.15 为 3 个设计方案的传输带宽。文献[35]为 8 通道在 PCIE 高速接口时的传输带宽, 与该条件比较, 本设计传输带宽提高近 206MB/s; 与文献[34]的最高传输带宽相比, 本设计有着 344.34MB/s 的提升, 与文献[35、34]相比, 本设计在单通道时有着较好的传输速度, 则由文献[29]可知本设计在 8 通道时依然有着较好的传输速度。

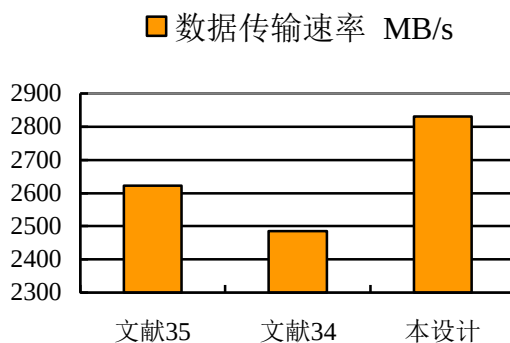


图 5.15 3 个设计方案的接口通路传输速度对比

5.5 本章小结

本章首先介绍了实验环境，后对 PCIE DMA 控制逻辑的各个模块和数据传输通道的整体进行仿真验证，然后对数据传输通道进行三方面的性能测试，最后本设计的传输速率与其它文献的传输速率进行对比。经分析验证，本设计的数据速率有大幅的提高，能满足软件无线电系统的高速传输。

第六章 总结与展望

6.1 工作总结

PCIE 是一项非常有应用前景的高速串行总线技术,为硬件层面的传输问题提供了新思路。本文以 PCIE 总线为研究对象,对数据传输通道进行设计,在设计过程中以 FPGA 开发环境用 Verilog HDL 硬件描述语言完成了 PCIE 接口逻辑、PCIE DMA 控制逻辑的 RTL 设计。基于 FPGA 实现的所有设计中,为解决现有方法存在的不足,提出了本文的设计方法及设计思路,并通过协议分析、FPGA 逻辑设计以及仿真与测试,证明了本设计的正确性和可行性。

本文以 Verilog HDL 语言为硬件程序的设计语言,ISE14.7 为 FPGA 逻辑设计的开发平台。本文首先对 PCIE 协议进行分析,提出了数据传输通道的解决方案,然后对数据传输通道进行自顶向下的模块化设计,利用 FPGA 实现了 PCIE 接口逻辑设计和 PCIE DMA 控制逻辑设计,其中通过 RTL 设计完成了 PCIE 接口逻辑,并在 PCIE DMA 控制逻辑的设计中为解决多个通道同时访问一块 DDR3 产生的冲突问题,设计并实现了基于轮询仲裁的缓存控制 DDR3 存储器,为提高数据传输带宽,设计并实现了一种命令缓冲机制的 DMA 引擎,为优化通道传输速率,设计并实现了基于信号带宽的动态优先级仲裁策略的通道仲裁器,最后使用 ISE14.7 并调用 Modelsim 对本设计进行仿真验证,功能验证无误后对本设计进行性能测试。经测试,数据传输通道在单通道和 8 通道时的最高传输带宽分别为 2325 MB/s 和 2830MB/s,本设计达到高速数据传输的效果,并有着较好的整体性能,实现了高效率的数据传输通道。此外,本文基于 FPGA 设计具有可硬件编程的优点,同时还具有稳定、实用、灵活等特征,可广泛应用于卫星遥测、无人机入侵数据获取、雷达系统等软件无线电平台的高速数据传输系统。

6.2 后期展望

本文设计并实现了一种基于 PCIE 总线的高速数据传输通道,综合考虑本设计的速率,在软件无线电中有较高的综合性能和应用场景。从本文的实验结果可知,数据吞吐量依然影响着数据传输的性能,即 PCIE 的传输速率可以继续提高,结合

PCIE 以及本文的工作进展，在未来工作中，本文至少还有两个方面的进步空间：

1、本文对 PCIE 采用 DMA 方式进行计算机与 FPGA 之间的高速传输，其每次 DMA 完成之后就要进行通道之间的切换，这样造成在数据传输的缓冲区随 DMA 传输长度增加也随着增加，且对于中断方式，本文没有深入探究，在后续的工作中，可在 DMA 传输时的通道仲裁和中断功能进行设计以使基于 PCIE 的数据传输通道的性能得到更好的发挥。

2、在后续的研究中，如果采用更高的操作系统和 PCIE 协议版本，针对实际需求开发高性能的驱动程序，数据传输速率将会得到更高的速率。

以上研究工作需要更多硬件设备，加之本人的科研能力有限，本文提出的基于 PCI Express 的高速数据传输通道的设计可能存在诸多亟需改进的地方，如有不足之处还请各位老师和同学给予谅解。

参考文献

- [1] Paun M. Very-low cost undersampling software radio receiver: A proof-of-concept implementation [J]. Electrical Engineering and Computer Science, 2019, 81(4): 181-188.
- [2] Akkaya S, Pehlivan I, Akgul A, et al. The design and application of bank authenticator device with a novel chaos based random number generator [J]. Journal of the Faculty of Engineering and Architecture of Gazi University, 2018, 33(3): 1171-1182.
- [3] Christiansen J M, Smith G E. Development and Calibration of a Low-Cost Radar Testbed Based on the Universal Software Radio Peripheral [J]. IEEE Aerospace and Electronic Systems Magazine, 2019, 34(12): 50-60.
- [4] 吴思博, 于宗光. 基于 CoreConnect 总线的 DMA 控制器设计 [J]. 半导体技术, 2020, 45(1): 31-36.
- [5] Tian W, Lei Q, Arun K S, et al. Energy-Efficient and Trustworthy Data Collection Protocol Based on Mobile Fog Computing in Internet of Things [J]. IEEE Transactions on Industrial Informatics, 2020, 16(5): 3531-3539.
- [6] Illawathure C, Parkin Gary, Lambot S, et al. Evaluating soil moisture estimation from ground-penetrating radar hyperbola fitting with respect to a systematic time-domain reflectometry data collection in a boreal podzolic agricultural field [J]. Hydrological Processes, 2020, 34(6): 1428-1445.
- [7] Manoufali M, Bialkowski K, Mobashsher A T, et al. In situ near-field path loss and data communication link for brain implantable medical devices using software-defined radio [J]. 2020, 68(9): 6787-6799.
- [8] De Moraes R S, De Freitas E P, Multi-UAV Based Crowd Monitoring System [J]. IEEE Transactions on Aerospace and Electronic Systems, 2020, 56(2): 1332-1345.
- [9] Y. Qian, F. Liang, H. Lu, et al. Design of a 10-gbps random number recorder [C]. 2016 IEEE-NPSS Real Time Conference (RT), Padua, Italy, 2016, 1-3.
- [10] Zhao X Y, Yang J Q, Peng Y M. Competitive and cooperative relationship evolution mechanism between urban rail transit and traditional bus [J]. Jilin Daxue Xuebao, 2017, 47(3): 756-764.
- [11] Bruno J S, Almenar V, Valls J. FPGA implementation of a 10 GS/s variable-length FFT for

- OFDM-based optical communication systems[J]. Microprocessors and Microsystems, 2019,64:195-202.
- [12] 刘娟,田泽,黎小玉,等. 一种 SoC 芯片中 PCIe 接口的 FPGA 平台验证[J].数字通信世界,2020,000(004):69-70.
- [13] Bai W, Wang N, Zhu J, et al. A generic framework for data acquisition and transmission[J]. Advances in Engineering Software, 2014, 68(03): 49-55.
- [14] Wei D, Li Y M. Generalized Sampling Expansions with Multiple Sampling Rates for Lowpass and Bandpass Signals in the Fractional Fourier Transform Domain[J]. IEEE Transactions on Signal Processing, 2016, 64(18):4861-4874.
- [15] Y. Liu, L. Hao, L. Meng, et al. Design of image processing system based on high speed datatransfer interface of pcie[J]. ence Technology and Engineering, 2019, 9(08): 32-29.
- [16] 美国国家仪器有限公司. Hr-100 宽带数字监测接收机顺利通过 [J]. 中国无线电, 2014, 6(11): 68-68.
- [17] 张芳菊. 无线接收机中高速 dma 数据传输通道的设计与实现 [D]. 成都: 电子科技大学.
- [18] Bross S I, Zalach H. Source Broadcasting and Asymmetric Data Transmission with Bandwidth Expansion[J]. IEEE Transactions on Communications, 2019, 67(10):6698-6710.
- [19] Weintraub W S, Spertus J A, Kolm P, et al. Effect of PCI on Quality of Life in Patients with Stable Coronary Disease[J]. Journal of Vascular Surgery,2008,48(5):1352-1352.
- [20] Neugebauer, Rolf, Audzevich, et al. Understanding PCIe performance for end host networking[C] //Proceedings of the 2018 Conference of the ACM Special Interest Group on Data Communication,2018:327-341.
- [21] Zhao J, Zeng X W, Guo Z C. Design and Implementation of High Speed PCIe Cipher Card Supporting GM Algorithms[J]. Journal of Electronics and Information Technology,2019,41(10): 2402-2408.
- [22] 肖明国,董明利,刘锋,等. 基于 PCIe 总线的数据采集卡设计与实现[J]. 计算机测量与控制, 2016, 24(3):252-254.
- [23] 杨亚涛,张松涛,李子臣,等. 基于 Zynq 平台 PCIE 高速数据接口的设计与实现[J].电子科技大学学报,2017,46(03):522-528.
- [24] Chen P W, Xing T H, Dong Y W, et al. Design of an Improved Scatter-Gather DMA Based on

- PCIe bus[J]. Radar Science and Technology, 2017, 015(005):558-562.
- [25] 何为,彭涛,栾辉,等. 基于 PCIe 总线的 DMA 控制器设计实现[J]. 信息技术, 2016(04):179-182.
- [26] Markussen J, Kristiansen L B, Borgli R J, et al. Flexible device compositions and dynamic resource sharing in PCIe interconnected clusters using Device Lending[J]. Cluster Computing, 2019,23(2):1211-1234.
- [27] Hossein K, Steffen M, Christian B, et al. High Performance FPGA-Based DMA Interface for PCIe[J]. IEEE Transactions on Nuclear Science,2014,61(2):745-749.
- [28] Jose F Z, Sergio L, Yury A, et.al. A PCIe DMA engine to support the virtualization of 40 Gbps FPGA-accelerated network appliances[C].2015 International Conference on ReConFigurable Computing and FPGAs,2015.
- [29] Rota L, Caselle M, Chilingaryan S, et al. A PCIe DMA arc-hitecture for multi-gigabyte per second data transmission[J]. IEEE Transactions on Nuclear Science, 2015, 62(3): 972-976.
- [30] 高俊,杨灿美,杜学亮. 基于 PCIe 总线的多路实时传输系统设计[J]. 电子技术应用, 2015(02):65-67.
- [31] 李文磊. 基于 PCIE 总线的高速数据传输系统的设计与实现[D]成都:电子科技大学.2016
- [32] 陈杨. 基于 PCIE 总线的高速数据采集系统设计与实现[D].杭州:浙江大学,2018.
- [33] Zhao Y X, Liu X, Yang Jiong. A resource and timing optimized PCIe DMA architecture using FPGA internal data buffer[J]. IEICE Electronics Express, 2018,16(1):1-12.
- [34] 母介滨. 基于 PCIE 总线的高速数据传输通道设计[D].成都:电子科技大学,2020.
- [35] 王之光,高清运. 基于 FPGA 的 PCIe 总线接口的 DMA 控制器的设计[J]. 电子技术应用, 2018, 44(1):9-12.
- [36] 陈沛伟,邢同鹤,董勇伟,等. 一种基于 PCIE 总线的改进分散集聚 DMA 的设计[J].雷达科学与技术, 2017, 015(005):558-562.
- [37] 丁维浩. 数据采集系统中 PCIE DMA 总线传输设计[D]. 2014.
- [38] 蒲恺,唐庆,田园,等. 基于 IP 核的 PCIE 总线接口逻辑的设计和实现[J]. 航空计算技术, 2017, 01(47):120-124.
- [39] Ladbury R, Berg M, Wilcox E, et al. Use of commercial FPGA-based evaluation boards for single-event testing of DDR2 and DDR3 SDRAM[J]. IEEE Transactions on Nuclear

- Science,2018,60(6):4457-4463.
- [40] 邹晨,高云. 基于 FPGA 的 PCIe 总线 DMA 传输的设计与实现[J]. 电光与控制, 2015,17(07):84-88.
- [41] 秦固平,毛瑞娟,杨小勇,等. 一种 PCIe 接口逻辑的 FPGA 实现[J]. 无线电工程, 2019, 49(04):91-96.
- [42] Nordyk S. PCIe digital I/O module speeds data acquisition[J]. Resuscitation,2013,84(1):78-84.
- [43] 李胜蓝,姜宏旭,符炜剑,等. 基于 PCIe 的多路传输系统的 DMA 控制器设计[J]. 计算机应用, 2017,37(3):691:694.
- [44] 张宇飞. 面向 Xilinx Virtex-7 的 DMA 数据传输软硬件系统设计实现及在 BFS 算法中的应用[D].长沙,国防科技大学 2015.
- [45] 闫海明,史岩,常于敏,等. 基于 WinDriver 的 PCIe 1553B 总线接口卡驱动设计与实现[J]. 信息通信, 2018, 185(05):104-105.
- [46] 杨亚涛,张松涛,李子臣,等. 基于 Zynq 平台 PCIE 高速数据接口的设计与实现[J]. 电子科技大学学报,2017,46(003):522-528.
- [47] Dongju, Lim, Manho, et al. A Parasitics-Induced Failure Mechanism for Transistors in the Bit-Line Sense Amplifier Region of DDP DDR3 DRAM During a CDM Event[J]. IEEE Transactions on Device and Materials Reliability, 2019,19(4):711-717.
- [48] Yang H Y, Kuo S H, Huang T H, et al. Random pattern generation for post-silicon validation of DDR3 SDRAM[J]. 2015:1-6.
- [49] Wang B, Du J, Bi X, et al. High bandwidth memory interface design based on DDR3 SDRAM and FPGA[C]// Soc Design Conference. IEEE, 2016.
- [50] 黄燕,黄光明.基 PXIe 高速 DAQ 性能提升的关键技术研究[J].电子测量技术, 2019, 42(02):136-140.
- [51] Turan F, Roy S S, Verbaauwhede I. HEAWS: An Accelerator for Homomorphic Encryption on the Amazon AWS FPGA[J]. IEEE Transactions on Computers, 2020, 69(8):1185-1196.
- [52] Lv H, Zhang S, Han W, et al. Design and Realization of a Aviation Computer Micro System Based on SiP[J]. Electronics, 2020, 9(5):766.
- [53] 伍小芹, 梁爽. 软件无线电中多速率信号处理设计及仿真[J]. 海南大学学报(自然科学版), 2017, 35(4):322-328.

- [54] Zhang Y, Tao L. Multi-Channel Data Acquisition System Based on FPGA and STM32[J]. Xibei Gongye Daxue Xuebao/Journal of Northwestern Polytechnical University, 2020,38(2):351-358.
- [55] Báscones D, González C, Mozos, D. An Extremely Pipelined FPGA Implementation of a Lossy Hyperspectral Image Compression Algorithm[J]. IEEE Transactions on Geoscience and Remote Sensing, 2020,58(10):7435-7447.
- [56] 赵参,李军,虞亚君. 基于 PowerPC 对 FPGA 上电自动加载的设计与实现[J].电子与封装,2020,020(002):38-42.
- [57] 何锐斌,李子扬,贺文静,等. 激光点云解算的软硬件协同设计与实现[J].电子技术应用,2019,45(03):106-109.
- [58] 李富国,宁凯,徐安俊. 基于 FPGA 的硬件测试电路设计[J]. 自动化与仪表, 2019, 034(002):35-37.
- [59] Pan W Q, Zheng F Y, Zhao Y. An efficient elliptic curve cryptography signature server with GPU acceleration[J]. IEEE Transactions on Information Forensics and Security, 2017 12(1):111-122.
- [60] 于东阳,苏彬.基于 Xilinx ISE 平台的 FPGA 电路设计[J]. 微处理机, 2012, 33(002):5-7.
- [61] 林坤. 基于 PCIe 的高速数据采集卡的 FPGA 设计与实现[D].成都:电子科技大学,2020.

攻读硕士学位期间完成的科研成果与获得奖励

一、读研期间发表的学术论文

- [1] 孙欣欣,李娟,田粉仙,等. 一种基于 PCIE 总线的 DMA 引擎研究[J]. 云南大学学报(自然科学版). doi: 10.7540/j.ynu.20200312 (第一作者)
- [2] Sun X X, Yang J, Li J, et al. Research on high-speed data acquisition system based on PCIE[J]. Advances in Intelligent Systems and Computing, 2021,1304:826-835. (第一作者)
- [3] Sun X X, Li J, Tian F X, et al. Design of FPGA Hardward based on Genetic Algorithm[C]// Proceedings of the 3rd International Conference on Computer Engineering, Information Science & Application Technology (ICCIA 2019). 2019. (第一作者)
- [4] 杨军,孙欣欣,张坤,等. “新工科”背景下 SOPC 实验课程的教学改革探讨[J]. 计算机教育,2020, 304(04):28-31. (第二作者, 导师第一作者)

二、读研期间申请的专利

- [1] 杨军, 孙欣欣,等. 一种 PCIE 高速数据采集系统的研究方法 申请时间: 2020 年 1 月 2 日, 申请号: 2020100241130, 实质审查阶段 (第二作者, 导师第一作者)

三、读研期间所获荣誉

- [1] 2019 年 12 月, 获云南大学硕士研究生学业奖学金二等奖
- [2] 2020 年 12 月, 获云南大学硕士研究生学业奖学金一等奖
- [3] 2019 年 07 月, 获第十四届中国研究生电子设计竞赛, 西南赛区三等奖
- [4] 2019 年 09 月, 获云南大学 2019 年第五届“互联网+”大学生创新创业大赛校级铜奖

四、读研期间申请的软件著作权

- [1] 基于 PCIE 的高速数据采集系统 V1.0, 2020SR0998603. (第一作者)
- [2] 基于 FPGA 的啸叫检测与抑制系统 V1.0, 2020SR0998438. (第一作者)
- [3] 基于 FPGA 异构加速的 Φ -OTDR 实时信号处理系统 V1.0, 2020SR0828086 (第一作者)

致谢

行文至此，学涯无尽，三载青春打马而过，三千往事浮现眼前，不觉感慨万千。

首先感谢杨军老师，本文是在杨军老师的指导下完成的。从刚开始对选题方向的迷惑到设计方案中的各种不确定因素再到论文的写作以及审定都得到杨军老师的关怀和教导。不管是科研上的难题还是为人处事上的道理，杨军老师都会以他多年的经验教育我，让我深受启发。衷心感谢杨军老师三年来的谆谆教诲与耐心指导。

在论文完成之际，同样感谢云南大学信息学院的各位老师，感谢各位老师在百忙之中给予指导和协助，并给予本论文提出宝贵的修改意见。

感谢计算机系统结构 3303 实验室的李克丽、梁颖、李娟和田粉仙以及孟圆、王圣凯、李俊和张晓对我生活和学习上的陪伴和帮助。

感谢已毕业的陈艳霜师姐、张坤师兄、毕方鸿师兄、田野师兄、王璞师兄和张玉明师兄对我学习和工作上给予的帮助和支持。

感谢室友毕鹏丽、王娟、李颖华以及 2018 级计科专硕的所有同学对我生活和学习上的关心。

由衷的感谢我的家人几十年来对我的照顾与帮助，更要感谢“岁月不饶人，我亦未曾绕过岁月”的自己。